WES 全流程分析路线图 · 矫正增强版（中文版）

A Practical Roadmap for Whole-Exome Sequencing Data Analysis — Corrected & Annotated Edition, in Chinese

目录

关于本版 · About this edition 2
每步包含什么 · What each step contains 3
问题清单速览 · Issue registry at a glance 3
0. 环境、目录与数据 · Environment, directories and data 5
0.1 所需工具 5
2.2 原理 7
2.3 实跑结果 7
2.4 坑与矫正 8
2.5 结果解读 8
2.6 调优与扩展 8
3. BWA-MEM 比对 · Alignment 8
3.1 原理 8
3.2 SAM → BAM、排序、索引（原版 3.2–3.3，逐字） 9
3.3 比对指标（原版 3.4，逐字） 9
3.4 实跑结果 9
3.5 调优与扩展 10
4. 标记 PCR 重复 · Mark PCR duplicates 10
4.1 原理 10
4.2 实跑结果 10
4.3 坑与矫正 11
4.4 调优与扩展 11
5. 碱基质量校准（BQSR）· Base quality score recalibration 11
5.1 原理 11
5.2 实跑结果 11
5.3 坑与矫正 12
5.4 调优与扩展 12
6. 胚系变异检测 · Germline variant calling 12
6.1 原理 12
6.2 实跑结果 13
6.3 坑与矫正 13
6.4 调优与扩展 13
7. 变异质量再校准（VQSR）· Variant quality score recalibration — 本版最大矫正点 13
7.1 原理 13
7.2 实跑结果 —— 两个头条发现 13
7.3 矫正方案 14
7.4 实跑命令说明 15
7.5 结果解读 16
8. ANNOVAR 功能注释 · Functional annotation 16
8.1 原理 16
8.2 命令（原版 8.1–8.3，逐字） 16
8.3 坑（问题 #2）：“富协议”块必挂——以及确切原因 16
8.4 坑（问题 #1 的下游）：翻倍的 VCF 对 ANNOVAR 做了什么 17

8.5 实跑结果 17
8.6 调优与扩展 17
9. 配对正常对照的体细胞突变与 TMB（Mutect2）· Somatic TMB 17
9.1 原理 17
9.2 坑与矫正 18
9.3 实跑结果 18
9.4 调优与扩展 19
10. CNVkit 拷贝数变异检测 · CNV detection 19
10.1 原理 19
10.2 命令（原版 10.2，逐字）与产出 19
10.3 坑与矫正（问题 #4） 23
10.4 结果解读 24
10.5 调优与扩展 24
11. 结果解读检查清单 · Interpretation checklist 24
12. 产物清单 · File summary 25
13. 总结与两轮对比 · Summary 26
复现说明 · Reproduction 27
附录 A. 问题登记表（完整版）· Issue registry 28
A1. VQSR MergeVcfs 翻倍 —— **重大** 28
A2. ANNOVAR 富协议库名不存在 —— **重大** 28
A3. 小变异集上的 VQSR —— **重大（统计性）** 28
A4. 杂交捕获 panel 上用 `-m amplicon` —— **重大** 28
A5. Mutect2 没用正常面板（PoN） —— 中等 28
A6. 数据路径不一致 —— 中等 28
A7. 每样本 33 GB 中间 SAM —— 中等（效率） 28
A8. TMB 计数口径不一致 —— 轻微 28
A9. 没有靶区深度指标 —— 轻微 29
A10. 命名不一致 —— 轻微 29
A11. 运行环境的坑（不算路线图 bug，但你一定会撞上） —— 信息 29
A12. 老师的参考结果偏离印刷版路线图 —— 重大（可比性） 29
附录 B. 一页矫正版命令速查 · One-page corrected pipeline 29

关于本版 · About this edition

本文档是课程路线图《WES 数据分析实战路线图》的**矫正增强版（中文版）**，与目录下的英文矫正版（`WES_roadmap_corrected.html/pdf`）内容一致、互为对照。它的特别之处在于：不是改出来的，而是**跑出来的**——先把原版路线图的命令**逐字**在计算集群上完整执行了两轮，把过程中遇到和预见的每一个坑都记录在案，再回头写这份矫正版。

轮次	数据集	靶区深度	耗时	目的
Demo 轮	`/lustre1/share/data_new`（OC 18.2 万 / PBMC 25.8 万读段对）	~0.5×	~25 分钟	机械地走通全部 13 步流程
全量轮	`/lustre1/share/data/OC_WES_all`（OC 3775 万 / PBMC 5251 万读段对）	**实测可用靶区深度 54.6× / 91.3×**	~4 小时（8 个 SLURM 作业，依赖链串联）	出真实结果，并与老师的 `analysis_full` 对拍

学习要点：每一步你都会看到”**原版怎么写 → 实际发生了什么 → 应该怎么改 → 为什么**”的完整链条。两轮实跑本身就是最好的对照组：0.5× 数据会暴露哪些步骤在”低深度下失效”，全量数据会暴露哪些步骤在”命令写法上就有问题”。

每步包含什么 · What each step contains

围绕原版命令，每个步骤都增加了六个学习板块：

1. **原理** —— 算法到底在做什么，用大白话讲清楚。
2. **命令** —— 原版路线图的命令（正确处逐字保留；**所有矫正都显式给出并说明理由**）。
3. **实跑结果** —— 两轮运行的实测数字（demo vs 全量）。
4. **坑与矫正** —— 我们踩到的每一个问题，全部附证据。
5. **结果解读** —— 数字怎么读、“好”长什么样。
6. **调优与扩展** —— 参数旋钮，以及这个工具在本路线图之外还能为你做什么。

问题清单速览 · Issue registry at a glance

完整登记表在**附录 A**。速览如下：

#	严重度	位置	问题	两轮实跑中的状态
1	**重大**	§7.3	两个 ApplyVQSR 全量输出做 `MergeVcfs` 会**把每条变异重复一遍**（每个输出都包含全部变异，合并后翻倍）	demo 实锤（14,782 → 29,564 条），**老师的 `analysis_full` 也中招（204,178 → 408,356 条）**
2	**重大**	§8.3	富注释协议引用的数据库在已装的 humandb 里不存在（`dbnsfp30a`、`exac03`、`gnomad_genome`、`avsnp147` vs 实际已装的 `dbnsfp54a`、`gnomad211_exome`、`gnomad41_exome`、`avsnp151`）	两轮均确认失败
3	**重大**	§7	2 样本、低深度的变异集上跑 VQSR，即使跑完也**统计上不可信**	demo “成功”跑完，但见 §7 的 tranches 分析
4	**重大**	§10	对**杂交捕获** panel（SureSelect V6）用 `cnvkit.py batch -m amplicon` —— 模式选错；另外 BED 无基因名列、未用 access 文件	能跑，但 demo 上产出全基因组范围的荒谬片段（cn 高达 6345）
5	中等	§9	Mutect2 没用**正常面板（Panel of Normals）**（集群上现成有 `1000g_pon.hg38.vcf.gz`）；群体频率资源用的是 1000G high-conf 而非推荐的 gnomAD AF-only	能跑，过滤力打折扣
6	中等	§0.2/§12	数据路径混乱：README 指向 `OC_WES`（已弃用的旧降采	三处引用互相矛盾；本版统一

#	严重度	位置	问题	两轮实跑中的状态
			样），新版路线图指向 `data_new`；旧版路线图指向一个**空目录**	
7	中等	§3	原版逐字执行要先落一个 33 GB 的 SAM 再排序——能跑，但每样本多占 ~33 GB、多花约一小时	全量轮我们改用 `bwa | sort` 管道（有记录的既定偏差）
8	轻微	§9	TMB 从 `somatic.filtered.vcf.gz` 里数 PASS，而 §9.2 另存了一个从未被使用的 `somatic.pass.vcf.gz`	口径不一致（数字相同）
9	轻微	§4/§11	整条路线图没有任何命令测量靶区覆盖度（flagstat 是全基因组的）——检查清单问了"深度够不够"，却没有命令能回答	本版补上 `CollectHsMetrics`（§11）——实测：OC 54.6×、PBMC 91.3×、demo 0.3×/0.5×
10	轻微	命名	`PBMC` 与 `blood` 混用（RG 写 `SM:blood`、目录叫 `PBMC`）；老师自己的输出目录拼写为 `2_vairant_call`	能用但易混淆；已记录
11	轻微	§5	`BaseRecalibrator` 跑全基因组（没加 `-L`）——不算错，WES 场景慢约 2 倍	两轮均按原文保留
12	**重大***	参考结果	**老师的参考结果本身就偏离了印刷版路线图：**单样本"联合"检出（blood gVCF 的 `SM` 标签是 `OC`，CombineGVCFs 把两样本塌缩成一个）且**没加 `-L`**（204,178 个位点中 72% 在靶区外），外加问题 #1 的翻倍（408,356 = 2×204,178）。*不是路线图文本的错，而是参考产物在执行时的偏离	从 VCF 头文件中读取实证；详见附录 A12，以免比较变异集时误判

学习要点：最重要的是 #1（VQSR 合并导致每条变异重复两遍，连老师的参考结果都带着这个 bug）和 #4（CNVkit 模式选错）。这两个都不影响"管线能不能跑完"，但**直接影响结果文件能不能用**——这正是"跑通了"和"跑对了"的区别。

0. 环境、目录与数据 · Environment, directories and data

0.1 所需工具

所有工具都来自集群上的共享 conda 环境：

source /lustre1/share/miniconda3/etc/profile.d/conda.sh
conda activate wes
```text

在计算节点（不是登录节点——见 §0.4）上核实的版本：fastp 0.23.2、BWA 0.7.19-r1273、samtools、GATK 4.6.2.0（Java）、CNVkit 0.9.11、ANNOVAR（2020 年代构建，带 hg38 humandb）、bcftools。

> **学习要点：** 集群上所有重计算都必须 `sbatch` 到 `cn-long` 分区做，登录节点只用来编辑文件和提交作业。本文所有实跑结果都来自 SLURM 作业（见 §13 复现说明）。

## 0.2 目录约定

原版路线图的环境变量块，逐字保留（这是新版与旧版 HTML 的**唯一**差异——旧版把 `$DATA` 指向了一个**空目录**，即问题 #6）：

```bash
export SHARE=/lustre1/share
export REF=$SHARE/references
export TARGETS=$REF/S07604514_AllTracks_V6_60_hg38.bed
export REFERENCE=$REF/hg38.fa
export DBSNP=$REF/dbsnp_138.hg38.vcf
export INDELS=$REF/Mills_and_1000G_gold_standard.indels.hg38.vcf
export HIGHCONF=$REF/1000G_phase1.snps.high_confidence.hg38.vcf
export HAPMAP=$REF/hapmap_3.3.hg38.vcf
export OMNI=$REF/1000G_omni2.5.hg38.vcf
export ANNOVAR=$SHARE/annovar_new
export HUMANDB=$ANNOVAR/humandb
export PATH=$ANNOVAR:$PATH
export DATA=$SHARE/data_new        # demo 轮用；全量轮用 $SHARE/data/OC_WES_all
export BASE=$HOME/wes_run_cluster  # 我们用了显式运行目录，见 §13
export OUT=$BASE
```text

**什么在哪里（全部经 `ls` 核实，不是想当然）：**

| 路径 | 内容 | 备注 |
| --- | --- | --- |
| `/lustre1/share/references/` | `hg38.fa` + BWA 索引 + `.dict`/`.fai`；GATK bundle（`dbsnp_138`、Mills、1000G high-conf、hapmap、omni）**且 `.idx` 全部在场**；靶区 BED `S07604514_AllTracks_V6_60_hg38.bed`；**`1000g_pon.hg38.vcf.gz` + `.tbi`（现成的正常面板!）**；`access-5kb.hg38.bed` | 只读共享区 |
| `/lustre1/share/data_new/` | demo FASTQ：`OC/OC_R1.fq.gz` 等（18.2 万对）、`PBMC/blood_R1.fq.gz`（25.8 万对） | 全量数据的 ~0.48% 抽样 ≈ 0.5× 靶区深度 |
| `/lustre1/share/data/OC_WES_all/` | 全量 FASTQ：OC 3775 万对（10.4 Gbp）、PBMC 5251 万对（15.0 Gbp） | 理想 ~95×/≈135×；**实测可用 54.6× / 91.3×**（§11） |
| `/lustre1/share/data/OC_WES/` | 旧降采样（体量相近，**内容不同**——md5 不一样） | 已弃用；README 仍指向这里（问题 #6） |
| `/lustre1/share/data/analysis_full/` | 老师的参考结果（全量数据）：比对、联合 VCF、VQSR、ANNOVAR | **只到 §8 为止**；没有体细胞/TMB 和 CNVkit 的参考；`4_cnv/` 是空的 |
| `/lustre1/share/annovar_new/humandb/` | hg38 数据库：`refGeneWithVer`、`cytoBand`、**`dbnsfp54a`、`gnomad211_exome`、`gnomad41_exome`、`avsnp151`** | 路线图 §8.3 富协议写的库名**不在**这里（问题 #2） |

**靶区：** BED 含 243,359 个区间、合计 **60.51 Mb**（这个数会出现两次：作为 `-L` 的区间，以及 §9.3 里 TMB 的分
```

母)。

0.3 参考基因组

hg38(`hg38.fa`),BWA 和 GATK 索引齐全。**开跑之前先检查**(我们逐一核实过都在——少一个 `.idx` 是"第一个作业必挂"的最常见原因):

```text
hg38.fa  hg38.fa.fai  hg38.dict
hg38.fa.{amb,ann,bwt,pac,sa}          # BWA 索引
dbsnp_138.hg38.vcf{,.idx}             # HaplotypeCaller -D 也要用
Mills_and_1000G_gold_standard.indels.hg38.vcf{,.idx}
1000G_phase1.snps.high_confidence.hg38.vcf{,.idx}
hapmap_3.3.hg38.vcf{,.idx}
1000G_omni2.5.hg38.vcf{,.idx}
```text

## 0.4 路线图(以及我们怎么在 SLURM 上跑)

路线图的各步以 sbatch 作业跑在 `cn-long` 分区(20 核节点、25 小时墙钟)。三个集群特有的坑,我们花了一小时才发现——记一次,永远受益:

1. **账号/QoS:** 作业必须写 `--account=bjx131_g1 --qos=bjx131cnl`(只写 `--qos=bjx131cnl` 不写账号会被拒:*"Invalid qos specification"*)。
2. **内存:** `cn-long` 节点向 SLURM 上报的 `RealMemory=1`,所以任何显式 `--mem=NN` 都会被拒(*"Memory specification can not be satisfied"*)。要用 **`--mem=0`**(语义是"占用节点全部内存")。
3. **conda 与 `set -u`:** 脚本若用 `set -u`,必须**先**激活 conda **再**打开它——conda 的 `activate.d` 脚本引用了未定义变量,会让作业一秒内死掉。

复现脚本都在各运行目录的 `scripts/` 下(见 §13)。

**两轮作业总览(全部在 `cn-long`,账号 `bjx131_g1`,QoS `bjx131cnl`,`--mem=0`):**

| 轮次 | 作业号 | 步骤 | 墙钟 | 结果 |
| --- | --- | --- | --- | --- |
| demo | 3048 | 3→10(逐字执行,含预期失败保护) | ~25 分钟 | 全部执行完;deviations 日志:1 条(富协议失败) |
| 全量 | 3049/3050 | 每样本 prep: fastp → `bwa\|sort` → index/flagstat → MarkDup → BQSR → HC-GVCF | 2 小时 20 分 / 3 小时 42 分(并行) | 双双干净 |
| 全量 | 3051 | 6.3→8 逐字(联合 → VQSR → MergeVcfs → ANNOVAR 基础) | 11 分钟 | 翻倍 bug 再现(§7.2) |
| 全量 | 3052 | 9 逐字(Mutect2 → 方向模型 → FilterMutectCalls → TMB) | 1 小时 13 分 | TMB = 1.78 mut/Mb |
| 全量 | 3053 | 10 逐字(`-m amplicon`) | 5.6 分钟 | 能跑;抗靶区基线为空(§10.2) |
| 全量 | 3054/3056→3061 | **矫正补充**: CollectHsMetrics ×4 样本 | 32 分钟 | 实测深度:OC 54.6× / PBMC 91.3×(§11) |
| 全量 | 3055 | **矫正补充**: CNVkit `-m hybrid` + access 文件 | 11 分钟 | 找到臂级事件(§10.3) |
| 全量 | 3056 | **矫正补充**: 顺序 ApplyVQSR + ANNOVAR 富协议(用实际存在的库名) | 9.6 分钟 | 零重复;富注释成功 |

*注意矫正作业(3055/3056)不只是"修复"——它们是课程的后半程:先按原文跑(3051/3053),再对同一份数据跑矫正命令,所以本版里的每个论断都是前后对照的实测。*

# 1. FASTQ 原始数据 · FASTQ files

## 1.1 检查 FASTQ

**原理。** 任何分析之前,必须先**看一眼**你的数据:读段数、读长、GC 分布。两个显而易见却常被低估的检查:(a) FASTQ 读长应与测序模式匹配(本文库是 2×137 bp 双端);(b) 每个循环的 GC 含量曲线——肿瘤 WES 里 GC 曲线有尖峰或漂移还算正常(捕获偏好 + 纯度所致),但曲线完全畸形通常意味着污染或接头穿透。

**命令**（原版逐字）：`zcat | head`、`wc -l / 4`，以及后面 fastp 的报告。集群上，`zcat file.fq.gz | head -8` 抽查；`zcat file.fq.gz | awk 'NR%4==2{n++; b=length($0)} END{print n, b}'` 数读段数×读长。

**实跑结果。**

| 样本 | 读段对（R1） | 读长 | 碱基数 | 60.51 Mb 靶区深度 |
| --- | --- | --- | --- | --- |
| demo OC（`data_new`） | 182,299 | 137 bp | 0.050 Gbp | 理想 ~0.82× → **实测可用 0.3×** |
| demo PBMC | 258,112 | 137 bp | 0.071 Gbp | 理想 ~1.2× → **实测可用 0.5×** |
| 全量 OC（`OC_WES_all`） | 37,746,701 | 137 bp | 10.41 Gbp | 理想 ~172× → **实测可用 54.6×** |
| 全量 PBMC | 52,508,649 | 137 bp | 14.97 Gbp | 理想 ~247× → **实测可用 91.3×** |

**结果解读。** "理想"列 = 总碱基 ÷ 靶区大小——小学算术就能得到的数。**实测**列是 CollectHsMetrics 报的 `MEAN_TARGET_COVERAGE`，扣掉了一切不能用的部分：脱靶碱基（20–9%）、重复（7–11%）、双端读段互相重叠的碱基（**25%——本文库插入片段峰值 ~150 bp，大多数读段对彼此重叠，见 §2.3 和 §11**）。理想深度与可用深度之间 3 倍的差距，是 WES 测序预算中最常见的误判来源。

> **学习要点：** 最值得记住的对比：全量 OC 理想深度 ~172×（10.41 Gbp ÷ 60.5 Mb），实测可用只有 **54.6×**——差了 3 倍多。差额去向：脱靶 ~20%、重复 ~7%、双端重叠 ~25%（文库插入片段峰值仅 ~150 bp，2×137 bp 的读段大量互相重叠，重叠部分只计一次）。demo 是全量的 0.48% 抽样，实测可用深度只有 0.3×/0.5×（**中位数为 0**）——低于任何可用的变异检测阈值，这就是后面体细胞检出为 0、CNV 全是噪声的根本原因。"理论深度"和"实测可用深度"的差距，本身就是 WES 设计中最重要的实务课。

# 2. fastp 质控与预处理 · Read quality control

## 2.1 生成质控输出

原版 fastp 块（逐字保留；`$OUT/0_fastq/` 同时放原始软链和 clean 输出）：

```bash
fastp \
  -i "$OUT/0_fastq/OC_R1.fq.gz" \
  -I "$OUT/0_fastq/OC_R2.fq.gz" \
  -o "$OUT/0_fastq/OC_R1.clean.fq.gz" \
  -O "$OUT/0_fastq/OC_R2.clean.fq.gz" \
  --detect_adapter_for_pe \
  --cut_right \
  --cut_right_window_size 4 \
  --cut_right_mean_quality 20 \
  --qualified_quality_phred 20 \
  --unqualified_percent_limit 40 \
  --n_base_limit 5 \
  --length_required 50 \
  --thread 8 \
  --html "$OUT/0_fastq/OC/OC.fastp.html" \
  --json "$OUT/0_fastq/OC/OC.fastp.json"
```

2.2 原理

- `--cut_right` 是 **3′ 滑窗修剪**（默认窗口 4、均值 Q20）：读段末端的碱基因 phasing/信号衰减而变差；先剪掉，免得日后在读段末端堆出一堆假阳性错配。
- `--detect_adapter_for_pe`：双端数据让 fastp 从读段重叠推断接头序列——比固定接头列表更灵敏。
- `--length_required 50`：修剪后的长度下限。太短的片段只会带来错配和重复假象。
- PE“overlap analysis”顺带免费给出**插入片段长度估计**——从 HTML 报告里读；它直接决定 §4.2 里重复率的解读。

2.3 实跑结果

Demo 轮（本版开工前已跑完）：

样本	修剪前后（读段数）	Q30（前 → 后）	插入片段峰值
demo OC	377,310 → 364,598	95.5% → 97.0%	**150 bp**
demo PBMC	525,176 → 516,224	96.9% → 97.8%	**150 bp**

全量轮：

样本	修剪前后（读段数）	Q30（前 → 后）	插入片段峰值
全量 OC	75,813,060 → 72,941,400	95.5% → 97.0%	150 bp
全量 PBMC	105,258,338 → 103,270,440	96.9% → 97.8%	150 bp

两个值得消化的观察：(a) 修剪几乎没删东西（3.8%/1.9% 的读段）——因为原始 Q30 已在 95% 以上，这是个健康文库；(b) 插入峰 **150 bp** 配上 137 bp 读长，意味着大多数读段对要**互相重叠 ~120 bp**。这部分重叠之后会被覆盖度统计丢弃（25% 的可用碱基—— §11）；如果你按读段数而非非重叠碱基估深度，就会虚高。两轮曲线一致——同一文库抽样的必然结果。

2.4 坑与矫正

- **(p1) 输出位置不一致：** 原版把 clean FASTQ 放在 `0_fastq/` 的根目录、报告放在 `0_fastq/OC/`。无害，但统一放 `0_fastq/OC/*` 会更整洁。我们按原文保留。
- **(p2) `--cut_right` vs `--cut_front`：** 对现代 137 bp 读段，5′ 端通常干净，只做 3′ 修剪是标准做法。原版的选择是对的——之所以记一笔，是因为很多教程画蛇添足加 `--cut_front`。
- **(p3) fastp 的 JSON 是机器可读的**——原版止步于 HTML；多样本的 **MultiQC** 汇总（见 §2.6）是”扩展”那一步。

2.5 结果解读

这类文库的”好”数字：修剪前 ≥90–95% 的碱基达 Q30、修剪/接头检测只动几个百分点、以及——最重要的——**重复率估计不在 fastp 的职责范围**（它来自 §4 的比对）。fastp 管的是读段层面的卫生；一切需要比对位置的指标都在后面。

2.6 调优与扩展

- `--thread` 近线性扩展；20 核节点上 8 线程很合适。
- UMI 文库用 `--umi`，不要依赖坐标重复（§4）。
- `--overlap_len_require`/`--overlap_diff_limit` 控制基于 PE 重叠的碱基矫正（`--correction`）——原版**没开**；低深度数据建议考虑加上。
- 多样本汇总：`multiqc 0_fastq/` 把所有 fastp JSON/HTML 收进一份报告。

学习要点：fastp 三个最常用开关——`--cut_right`（滑窗去低质量尾）、`--detect_adapter_for_pe`（利用 PE 重叠自动识别接头）、`--length_required`（太短的读段宁可不要）。报告里最该看的三处：Q30 比例、接头残留比例（应为 0 或接近 0）、插入片段分布峰（应显著大于读长 137 bp，否则有大量重叠读段）。

3. BWA-MEM 比对 · Alignment

3.1 原理

BWA-MEM 先用精确匹配种子（MEM）定位，再做局部比对延伸；soft-clip 的末端处理接头/部分重叠。几个你必须能讲清楚的概念：

- **读段组（RG）**是每条读段的”出身”头信息：`ID`（测序 lane）、`SM`（**样本**——GATK 用它来分组读段，SM 错 = 基因分型错）、`LB`（**文库**—— §4 重复估计的单位）、`PL` 平台。一个 BAM 一个 SM；一个 SM 可以有多个 LB。
- **`-M`**：把分裂命中标为 secondary（而非 supplementary），为 Picard 兼容——如今只是惯例，无害，照原版保留。
- **`-t 8`**：线程数。BWA-MEM 在 ~16 线程内近线性加速。

命令（原版逐字——OC 所示；PBMC 相同，用 `SM:blood`、`LB:WES_PBMC`）：

```
bwa mem -t 8 -M \
  -R "@RG\tID:OC\tPL:illumina\tSM:OC\tLB:WES_OC" \
  "$REFERENCE" \
  "$OUT/0_fastq/OC_R1.clean.fq.gz" \
  "$OUT/0_fastq/OC_R2.clean.fq.gz" \
  > "$OUT/1_alignment/OC/OC.sam"
```

3.2 SAM → BAM、排序、索引（原版 3.2–3.3，逐字）

```
samtools view -bS -@ 8 "$OUT/1_alignment/OC/OC.sam" > "$OUT/1_alignment/OC/OC.bam"
samtools sort -@ 8 -m 4G "$OUT/1_alignment/OC/OC.bam" -o "$OUT/1_alignment/OC/OC_sorted.bam"
samtools index -@ 8 "$OUT/1_alignment/OC/OC_sorted.bam"
```

坑与矫正（问题 #7）。 按字面执行会先落一个 **33 GB 的 SAM**（全量数据），再复制成 BAM，再排序出 sorted BAM——同一份信息在磁盘上存三遍，多花约 ⅓ 墙钟。标准做法（也是我们全量轮实际跑的）是管道：

```
bwa mem -t 8 -M -R "@RG\tID:OC\tPL:illumina\tSM:OC\tLB:WES_OC" \
  "$REFERENCE" R1.clean.fq.gz R2.clean.fq.gz \
  | samtools sort -@ 8 -m 4G -o OC_sorted.bam -
```

产出的 `OC_sorted.bam` **内容逐字节等价**——同样的记录、同样的顺序（先染色体再位置）。区别只是中间文件从未存在过。这是全量轮唯一一处刻意的效率偏差，已记录在该运行的 `log/deviations.log`。

> **学习要点：**为什么要排序？下游所有工具（去重、BQSR、HaplotypeCaller、CNVkit）都要求 BAM 按”染色体 → 位置”排好，才能顺序流式读取。SAM→BAM 是压缩（文本→BGZF 二进制，约省 4–6 倍空间）；管道化的意义是让”比对输出 → 排序输入”直接在内存里传递，省掉 33 GB 中间 SAM——结果完全一样，磁盘和时间都省一半以上。

3.3 比对指标（原版 3.4，逐字）

```
samtools flagstat -@ 8 "$OUT/1_alignment/OC/OC_sorted.bam" > "$OUT/1_alignment/OC/OC.flagstat.txt"
```

3.4 实跑结果

Demo 轮（0.5×）：

指标	OC	PBMC
总读段	397,780	517,164
Primary mapped	364,582 (100.0%)	516,221 (100.0%)
Properly paired	83.29%	99.46%
Secondary（分裂）比对	33,182（primary 的 9.1%）	940 (0.18%)

全量轮：

指标	OC	PBMC
总读段	79,607,191	103,455,092
Primary mapped	72,941,400 (100.00%)	103,270,440 (100.00%)
未比对	**311**	1,594
Properly paired	**83.22%**	99.44%
配对落在不同染色体	11,526,594（**15.8%**）	444,126 (0.43%)
Secondary（分裂）比对	6,665,791 (9.1%)	184,652 (0.18%)

结果解读。

- 两轮都是”100% 比对”——不是抽样巧合：全量轮 79.6M 条读段里只有 311 条未比对。**这套教学数据在发布前就剔除了未比对读段**（预过滤数据集）。新鲜实验室数据通常是 96–99%；**低于 90% 就要怀疑污染或物种错配**。
- OC 的 **properly paired 83.2%**——**两轮完全一致**——**是肿瘤文库的真实属性，不是抽样伪象**：15.8% 的 OC 读段对的另一端落在另一条染色体上（PBMC 仅 0.43%）。这个量级是重度重排肿瘤基因组（HGSOC 携带大规模结构变异）在比对层面的签名。PBMC 的 99.4% 才是正常基因组的样子。
- OC 的 secondary 比对 9% vs PBMC 0.18%：secondary/supplementary 读段提示重复序列含量（肿瘤基因组 + 外显子捕获探针落在重复区）。知道即可，不必恐慌。

学习要点： flagstat 三问：① 比对率——本数据集 demo 和全量都是 100%，这不是抽样巧合，而是教学数据发布前就剔除了未比对读段（全量也只有 311/79.6M）；新鲜实验室数据应为 96–99%；② properly paired——**OC 的 83.2% 在两轮中完全一致，是肿瘤样本的真实属性而非抽样伤**：15.8% 的 OC 读段配对到不同染色体（PBMC 仅 0.43%），这是重度重排肿瘤基因组（HGSOC 典型特征）在比对层面的签名，也是下游 §10 结构性 CNV 的伏笔；③ secondary/supplementary——肿瘤样本 9% vs 正常 0.18%，重复序列含量差异，外显子组里几个百分点属正常。

3.5 调优与扩展

- 线程：空闲的 20 核节点上 `-t 16` 可减半墙钟；`-K`（块大小）对超大参考基因组有帮助。
- `bwa mem -v` 看详细日志；`-Y`（soft-clip supplementary）配合的是 GATK IndelRealigner 时代的老工具——现代 GATK 不在乎。
- **脱靶率**：`samtools view -c -L $TARGETS bam` ÷ 总数——捕获式 WES 信息量最大的单项 QC（直接决定 §11 的覆盖度问题）。
- 嘈杂/古 DNA：`bwa mem -B`（批次大小，降低内存）和调整打分。
- **MultiQC** 也吃 flagstat：所有样本跑完后 `multiqc run_dir/`。

4. 标记 PCR 重复 · Mark PCR duplicates

4.1 原理

“重复” = **5′ 坐标和方向与另一读段对完全一致**的读段对。随机打断时真分子几乎不可能撞出同一坐标；PCR 扩增则极易。重复是**标记、不删除**：GATK 统计工具忽略它们，但它们留在 BAM 里（可审计，`samtools view -F 1024` 随时找回）。

为什么**必须在排序之后**：MarkDuplicates 需要坐标顺序才能流式地看到同位置的读段对。这也解释了 **LB 标签为什么重要**——重复只在同一个文库内才**应当**出现；同一样本的两个文库是两次独立打断，跨文库标记是错的。

命令（原版 4.1，逐字）：

```
gatk --java-options "-Xmx8g" MarkDuplicates \
  -I "$OUT/1_alignment/OC/OC_sorted.bam" \
  -O "$OUT/1_alignment/OC/OC_sorted.markdup.bam" \
  -M "$OUT/1_alignment/OC/OC_markdup_metrics.txt" \
  --CREATE_INDEX true
```

4.2 实跑结果

Demo 轮： 重复率 OC **0.033%**、PBMC **0.06%**——基本为零，这正是 0.48% 抽样的必然结果（一对重复的两个成员同时存活于抽样中的概率 ~$0.48\%^2 \approx 0.002\%$）。

全量轮（数字全部来自 metrics 文件的 per-library 行）：

样本	检查读段对	重复对	光学重复	PERCENT_DUPLICATION	文库复杂度估计
OC	36,468,418	2,583,317	1,853,690（占重复的 71.8%）	**7.08%**	809,511,121

样本	检查读段对	重复对	光学重复	PERCENT_DUPLICATION	文库复杂度估计
PBMC	51,634,767	5,562,361	3,192,510 (57.4%)	**10.77%**	478,822,815

老师的全量运行（`analysis_full`）：OC **7.39%**、PBMC **11.01%**——我们的数字与两者都在 0.4 个百分点以内。这个吻合度是我们 prep 链（fastp → 管道化 bwa|sort → MarkDup）忠实复现参考 prep 的最强单点验证。

结果解读。~100× 的杂交捕获 WES，重复率预期在 **5–20%**。重复率持续攀升 = 文库复杂度收缩（模板制备差、过度扩增）；metrics 文件的 `ESTIMATED_LIBRARY_SIZE` 才是诚实的指标（数亿到数十亿 = 健康）。光学重复一栏还有个细节：**OC 的重复有 71.8% 是光学重复**（flowcell 上相邻来源），这通常指向 patterned flowcell 的聚类模式，而非 PCR 过度扩增——是测序硬件属性，不是文库失败。

4.3 坑与矫正

- **(p1) 光学重复 vs PCR 重复：** metrics 会分开统计。光学重复来自 flowcell 相邻簇；若光学 >> 非光学，怀疑 patterned flowcell / 上样浓度，而不是文库制备。
- **(p2) "Unknown Library"** 出现在老师的 metrics 里（他们的 RG 没写 `LB`）；按原版的 RG 写法会得到具名行。保留 `LB`——重复统计按文库分列。
- **(p3) UMI 文库要用** UMI 感知的重复标记（GATK `UmiAwareMarkDuplicatesPoN` 或 fgbio `CallMolecularDuplicates`）；纯坐标标记在那里会低估复杂度。

4.4 调优与扩展

- 大 BAM 记得 `--TMP_DIR` 指向 scratch 盘（MarkDuplicates 临时文件很大）。
- 老 BAM 因 NM/MD 标签顺序报错时可用 `--VALIDATION_STRINGENCY SILENT`——更好的做法是修 BAM。
- `samtools markdup`（配合 `samtools fixmate`）是更快的替代；GATK 的 `EstimateLibraryComplexity` 只凭读段对就能预测复杂度。

5. 碱基质量校准（BQSR）· Base quality score recalibration

5.1 原理

仪器给的 Phred 分数是**估计值**，按循环、机器、上下文系统性偏移。BQSR 从"我们**知道**为真的位点"（已知变异资源）上的错配学习一个经验误差模型，然后改写质量分数。心智模型：如果已知参考位点上的碱基读出在上下文 C 里看起来像 Q35，那么以后 C 上下文里就按 Q35 对待，不管仪器说什么。

命令（原版 5.1–5.2，逐字——known-sites 三件套 = dbsnp + Mills + 1000G high-conf，本集群上 `.idx` 全齐）：

```
gatk --java-options "-Xmx20g -XX:ParallelGCThreads=4" BaseRecalibrator \
  -R "$REFERENCE" \
  -I "$OUT/1_alignment/OC/OC_sorted.markdup.bam" \
  --known-sites "$DBSNP" \
  --known-sites "$INDELS" \
  --known-sites "$HIGHCONF" \
  -O "$OUT/1_alignment/OC/OC.recal_data.table"

gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" ApplyBQSR \
  -R "$REFERENCE" \
  -I "$OUT/1_alignment/OC/OC_sorted.markdup.bam" \
  --bqsr-recal-file "$OUT/1_alignment/OC/OC.recal_data.table" \
  -O "$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam"
```

5.2 实跑结果

Demo：recal 表 108 KB / BQSR BAM 54 MB；每样本约 2.5 分钟。 全量轮：recal 表每样本 110 KB（24 个协变量行 × ~400 个上下文行——模型形状与 demo 相同，只是计数更密），BQSR BAM 8.8 GB（OC）/ 8.2 GB（PBMC）；作业日志中的耗时——BaseRecalibrator ~31 分钟（OC）/ ~43 分钟（PBMC），ApplyBQSR ~20 分

钟 / ~28 分钟。注意 BQSR BAM 是 markdup BAM 的 ~1.9 倍：ApplyBQSR 改写了每条读段的每个质量值，重压缩的 BAM 未必缩小——保留所有中间文件时这是个容易漏算的磁盘开销。

结果解读。recal 表的 `EmpiricalQuality` vs `EstimatedQ` 两列是重点：1–5 个 Q 的系统性偏差属正常且值得校准；BQSR 的**报告图**（`gatk AnalyzeCovariates`）让它可视化。本课两个 BQSR 真相：(a) dbsnp **加上** Mills + 1000G 作 known-sites 时，真实的新变异几乎不受影响——模型"相信"这三件套；(b) 0.5× 数据上 BQSR 照样能跑，但每个协变量格子的经验计数薄得可怜，校准基本是噪声——又一个"机械上能跑、统计上无意义"的案例。

5.3 坑与矫正

- **(p1) 没加 `-L` 限制：**BaseRecalibrator 扫描整个基因组，尽管下游只关心外显子组。加 `-L $TARGETS` 能把 WES 墙钟砍近一半。我们按原文保留（不算错，只是慢）。
- **(p2) BQSR 需要足够的数据量。** 靶区碱基 <10^7 时模型会抖。
- **(p3) 序列字典必须匹配**（本集群 `hg38.dict` 在场——没问题），否则别做 BQSR。

5.4 调优与扩展

- `--native-pair-hmm-threads` 是 HaplotypeCaller 的参数，与这里无关；BaseRecalibrator 的并行靠 GC 线程 +（较新 GATK 的）`--parallelism`。
- 画协变量图：`gatk AnalyzeCovariates -bqsr recal_data.table -O report.pdf`——整个步骤最好的教学产物。
- 平台研究充分时某些管线会跳过 BQSR；只要错配模型显示有偏，GATK 仍然建议做。

6. 胚系变异检测 · Germline variant calling

6.1 原理

HaplotypeCaller（HC）不是逐 pileup 的 SNP 检测器：它 (1) 找到有变异证据的活跃区域，(2) 局部**重组装单倍型图**，(3) 对每条读段与每条候选单倍型跑 pair-HMM 似然，(4) 输出最可能的基因型。这正是它抓短读段附近 indel 的能力远超 pileup 检测器的原因。

GVCF 模式（`-ERC GVCF`）：对每一块（变异或纯参考）都输出置信区间。代价：文件更大。收益：**可扩展的联合基因分型**——之后加第 31 个样本，只需重跑 `CombineGVCFs`/`GenotypeGVCFs`，不用重跑 HC。这套两样本"GVCF 工作流"正是扩展到整个队列的模式。

命令（原版 6.2–6.4，逐字）：

```
gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" HaplotypeCaller \
  -ERC GVCF -R "$REFERENCE" \
  -I "$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam" \
  -D "$DBSNP" -L "$TARGETS" \
  --native-pair-hmm-threads 8 \
  -O "$OUT/1_alignment/OC/OC.g.vcf"

gatk --java-options "-Xmx16g -XX:ParallelGCThreads=4" CombineGVCFs \
  -R "$REFERENCE" \
  -V "$OUT/1_alignment/OC/OC.g.vcf" \
  -V "$OUT/1_alignment/PBMC/blood.g.vcf" \
  -O "$OUT/2_variant_call/combined.g.vcf"

gatk --java-options "-Xmx16g -XX:ParallelGCThreads=4" GenotypeGVCFs \
  -R "$REFERENCE" \
  -V "$OUT/2_variant_call/combined.g.vcf" \
  -D "$DBSNP" \
  -O "$OUT/2_variant_call/OC_blood_variants.vcf"
```

原版 6.4 还有可选的 VariantAnnotator（用 OC 的 BAM）——它注释基于覆盖度的字段；GATK4 里过滤之前**不再需要**它，原版自己也是这么说的。

6.2 实跑结果

Demo 轮： 联合变异集 **14,782 条变异记录**（13,632 个 SNP + 1,151 个 indel，另少量混合）——肿瘤/正常对联合基因分型；PBMC 的胚系变异也出现在肿瘤列里。

全量轮： 联合变异集 **55,123 条变异记录**（OC+blood，双样本、限定靶区：HC/Combine/Genotype 都带 `-L $TARGETS`）。

老师的参考：204,178 条。**小心天真的比较——这两个数不是同一个量**（见附录 A12，问题 #12）。对老师 VCF 的实测结论：(a) 它只有**一个样本**（只有 OC——他们的 blood gVCF 的 RG `SM` 标签写成了 `OC`，CombineGVCFs 遇样本重名把两者塌缩成一个）；(b) 它是**全基因组范围**检出的——HaplotypeCaller/GenotypeGVCFs **都没加 `-L`**——其 **72% 的位点落在捕获靶区之外**（大量位于低复杂度/脱靶、浅覆盖区）。我们的 55,123 是双样本、靶区内变异集；他们的 204,178 是单样本、全基因组变异集。深度是主要差异，但范围和样本数同样要紧——比较变异集之前永远先读 VCF 头（`##GATKCommandLine`、`#CHROM` 行）。

对自己数字做个 sanity check：60.51 Mb 外显子组按每 kb 约 1.2–1.4 个胚系变异 → 每样本约 75–85k；两个不同个体、共享约 70% 的常见变异 → 去重后约 55–65k 个位点。55,123 正落在这个窗口里。

6.3 坑与矫正

- **(p1) `-D dbsnp` 只加 RS ID 注释**——它**不**限制检出范围。常见误解是”dbsnp 告诉 HC 该找什么”；不是。
- **(p2) 千万别把这个联合 VCF 当体细胞变异集**——这是**两个个体**的胚系变异。原版的点睛之笔（也是 §9 的出发点）正是这一点。
- **(p3) PBMC 当”正常对照”**用于配对体细胞检出是可接受的，但它毕竟是另一种组织，有自己的嵌合/衰老突变；血液里的克隆性造血（CHIP）变异**会**出现在这里——现代免疫肿瘤学的热门话题。

6.4 调优与扩展

- `--native-pair-hmm-threads 8` 是我们实跑中单项提速最大的参数——永远用它。
- 队列场景：~50 样本以上 `GenomicsDBImport` 扩展性优于 CombineGVCFs（2 个样本用 CombineGVCFs 没问题）。
- `--interval-padding 50` 能捞回 BED 边界外剪接区的变异——几乎零成本，临床上很值。
- QUAL 的解读：它是 Phred 尺度的**基因型**置信度，不是变异频率。一个 QUAL 5,000 的杂合子意思是”P(错) ≈ 10^{-500}”——构造上就过度精确。

7. 变异质量再校准（VQSR）· Variant quality score recalibration — 本版最大矫正点

7.1 原理

硬过滤用固定阈值（QD<2 → 差）。**VQSR 则是在你的变异集上训练高斯混合模型**：用”真相”资源（SNP 用 hapmap/omni、indel 用 Mills）当正样本标签，dbsnp 当”已知但别全信”群体。每条变异得到一个 **VQSLOD** 分数（给定其注释向量 QD、MQ、FS、SOR、ReadPosRankSum、MQRankSum，它是”真变异 vs 假 artifacts”的对数几率）。你保留一个真实敏感度分层（tranche；此处 SNP 99.5%、indel 99.0%），把分层外打分最差的部分过滤掉。

不可协商的前提：训练数据要够。 GATK 的指导值：约 30 个 WGS 样本（或大队列变异集；外显子组因为每样本变异更少，需要的样本比 WGS 更多）。2 个外显子组、14k 变异，模型数学上可拟合但统计上是虚构的——它会毫不抗议地给你产出一份 tranches 文件。

7.2 实跑结果 —— 两个头条发现

发现 A：VQSR 在 demo 上”成功”了——而这恰恰是个陷阱。 两个 VariantRecalibrator 都在 14,782 条变异的 demo 集上跑完了。但看看它到底学到了什么：

证据	Demo（0.5×，14.8k）	我们全量（55.1k，双样本靶区内）	老师（204k，全基因组）	健康 WES 预期
可用真相位点（hapmap+omni ∩ 变异集）	7,388	28,324	85,159	随队列增长
SNP minVQSLod（99.9% tranche）	−2.56	−11.62	−2.47	温和的负尾
SNP minVQSLod（100% tranche）	**−5889.9**	−1111.4	−25.5	有界尾部
novel Ti/Tv（90% tranche）	**2.46**（紧贴 known 的 2.71）	2.50（known 2.63）	**1.69**（known 2.34）	novel ≈ 1.5–2.0，明显低于 known
模型文件大小	1.2 MB	4.3 MB	15.1 MB	—
Indel known Ti/Tv	0.0000（indel 模式的正常伪象）	0.0000	0.0000	n/a

demo 模型的失效模式体现在两个数字上。其一，100% tranche 的截断值 −5889 意味着模型**在尾部毫无判别力**——它在噪声里分离噪声。其二，更微妙：健康变异集里 novel（不在 dbsnp 里的）变异富集 artifacts，所以 **novel Ti/Tv 应明显低于 known Ti/Tv**——老师的运行正是标准样子（1.69 vs 2.34）。demo 的 novel Ti/Tv 却是 2.46，**贴着** known 值（2.71）：模型已无力区分”最差”和”最好”的变异，因为 0.5× 深度下连真相位点都只有 1–2 条读段支撑。我们自己的全量轮居中（novel 2.50 vs known 2.63）：55k 靶区变异上模型可用了，但按 GATK 自己”≥30 样本”的标准仍然偏薄。

> **学习要点：** VQSR 在 14,782 个变异上”能跑完”不等于”能用”。三个铁证：① 训练用的 truth 位点只有 7,388 个（全量轮 28,324、老师 85,159）——模型是在极稀疏的标签上拟合的；② 100% tranche 的 minVQSLod = −5889——健康数据的尾部应在 −10 到 −30 之间，−5889 意味着模型在噪声里找规律；③ 最微妙的一点：健康数据里 novel Ti/Tv 应显著低于 known（老师的 1.69 vs 2.34 就是标准样子），而 demo 的 novel 2.46 几乎贴着 known 2.71——模型已无力区分”最差”和”最好”的变异，因为 0.5× 深度下连 truth 位点都只有 1–2 条读段支撑。这不是”VQSR 挂了”，而是”VQSR 拟合了一个没有统计意义的东西”——比挂了更危险，因为它不报错。

发现 B（问题 #1）：原版 §7.3 的 MergeVcfs 把每条变异都重复一遍。 `ApplyVQSR --mode SNP` 输出的**不是只有 SNP**——它输出**整个 VCF**，只是套用了 SNP 模型打分（indel 原样透传、未过滤），`--mode INDEL` 反之。把两个输出合并，就得到**每条变异两份**：

变异集	原始联合记录	原版 §7.3 MergeVcfs 之后	重复位点
demo	14,782	**29,564**	**100%（14,782/14,782）**
老师 `analysis_full`	204,178	**408,356**	**100%**

老师自己的参考产物也带着同样的重复——证明这是路线图的 bug，不是环境特例。具体损害（两轮都实测过）：每个位点出现**两次**、带两个不同的 FILTER 值——SNP 模式的记录带着 SNP 模型的裁决（`PASS` 或 `VQSRTrancheSNP…`），INDEL 模式的记录对 SNP 显示 `.`（透传未过滤），反之亦然。两个后果：(a) **原始记录数翻倍**（29,564；110,246；408,356）——下游一切”按行数数”的统计（包括 ANNOVAR 的 avinput，§8）全部翻倍；(b) “PASS” 计数碰巧保持正确（我们全量轮为 53,988），因为每个位点恰有一条 SNP 模式裁决——这个意外的软点让 bug 在随手 QC 时隐身。全量轮还暴露一个细节：SNP 模型把某记录过滤掉时，它的孪生另一条仍是 `.`——于是 `SelectVariants` 式”丢弃非 PASS”的逻辑会**悄悄留下未过滤的孪生记录**。

7.3 矫正方案

两种标准修法；**修法 1 是 GATK 官方最佳实践**（顺序套用——每条变异只被它自己类型的模型过滤一次）：

修法 1 —— 顺序 ApplyVQSR（推荐）：

```
# SNP 模型 → 应用到完整变异集
gatk VariantRecalibrator ... -mode SNP -O snps.recal --tranches-file snps.tranches ...
gatk ApplyVQSR -V raw.vcf -O step1.vcf --mode SNP \
  --recal-file snps.recal --tranches-file snps.tranches --truth-sensitivity-filter-level 99.5

# INDEL 模型 → 应用到已过 SNP 滤的输出
gatk VariantRecalibrator ... -mode INDEL -O indel.recal --tranches-file indel.tranches ...
gatk ApplyVQSR -V step1.vcf -O all.VQSR.vcf --mode INDEL \
  --recal-file indel.recal --tranches-file indel.tranches --truth-sensitivity-filter-level 99.0
```

修法 2 —— 拆分、过滤、合并（SelectVariants → 各自 ApplyVQSR → MergeVcfs），如果你想要按类型分盘的 VCF。

以及本课数据（2 样本）的诚实替代方案：硬过滤。 GATK 自己对小变异集的建议就是 `VariantFiltration`：

```
gatk VariantFiltration -V OC_blood_variants.vcf -O filtered.vcf \
  --filter-name "SNP_QD2"     --filter-expression "QD < 2.0 && SNP" \
  --filter-name "SNP_MQ40"    --filter-expression "MQ < 40.0 && SNP" \
  --filter-name "SNP_FS60"    --filter-expression "FS > 60.0 && SNP" \
  --filter-name "SNP_SOR3"    --filter-expression "SOR > 3.0 && SNP" \
  --filter-name "INDEL_QD2"   --filter-expression "QD < 2.0 && INDEL" \
  --filter-name "INDEL_FS200" --filter-expression "FS > 200.0 && INDEL" \
  --filter-name "INDEL_SOR10" --filter-expression "SOR > 10.0 && INDEL"
```

VQSR 在队列规模（30+ 外显子组）才有意义；1–10 个样本用硬过滤。**这是整个步骤里最可迁移的一课。**

> **学习要点：**矫正方案记忆点：**顺序套用**（SNP 模型套完的输出，再套 INDEL 模型）——每条变异只被它自己类型的模型过滤一次；或者**先拆再合**（SelectVariants 按类型拆开 → 各自 ApplyVQSR → MergeVcfs）。而如果只有 1–10 个样本，GATK 官方建议干脆不用 VQSR，改用硬过滤（上面的 QD/MQ/FS/SOR 阈值就是 GATK 文档的标准值）。VQSR 的适用边界：约 30 个以上样本、或 callset 足够大——“能跑”≠“该用”。

7.4 实跑命令说明

两轮里我们都按原版 §7.1/7.2 逐字跑了 VariantRecalibrator/ApplyVQSR（recalibrator + 按模式 ApplyVQSR，用原版的 99.5/99.0 档），§7.3 的 MergeVcfs 也逐字跑了——为的是**用实测数字记录**翻倍现象（demo 14,782 → 29,564；全量 55,123 → 110,246），而不是口头断言。最终交付用的矫正文件由修法 1（顺序 ApplyVQSR）从同一组 recal 模型产出。

全量轮数字，按原文跑 vs 矫正：

量	原版 §7.3 原样（MergeVcfs）	矫正后（顺序 ApplyVQSR，作业 3056）
总记录数	**110,246**（= 2 × 55,123）	**55,123**（每个位点一条）
唯一位点数	55,123	55,123
PASS 记录	53,988	53,988
带 .（未过滤）FILTER 的位点	53,988	**0**
被过滤记录	56,258（绝大多数是未过滤孪生）	1,135（696 SNP-tranche + 427 + 12 INDEL-tranche）
由此产出的 ANNOVAR avinput 行数	110,246	55,123

全量轮的 tranches（作业 3051）：SNP 真相位点 28,324，99.9% 档 minVQSLod −11.62 / 100% 档 −1111；100% tranche 的 novel Ti/Tv 1.65——模型行为像一个真实（尽管偏薄的）变异集模型。顺序套用随后从 55,123 条里恰好移除 1,135 条（2.1%）——对一个干净的 WES 变异集来说这是合理过滤率，与 demo 的退化 tranches 形成鲜明对照。

7.5 结果解读

- **tranches 文件**是质量仪表盘：分层越放越宽（99 → 99.9 → 100）时，每档的 `novelTiTv` 应当下降。如果它上升或抖动，说明模型在过拟合——demo 里肉眼可见。
- `--truth-sensitivity-filter-level` 的语义："至少保留 X% 的真相位点变异"；SNP 用 99.5%（更松，SNP 模型更强）、indel 用 99.0%（indel 真相更稀缺，稍严格来补偿）。
- VQSLOD 分布：`bcftools query -f '%VQSLOD\n' vcf | sort -g | awk ...`——健康的运行分布平滑；我们的 demo 是一个退化尖峰。
- **不要在两轮之间直接比较原始 QUAL；比较 tranches 行为。**

8. ANNOVAR 功能注释 · Functional annotation

8.1 原理

VCF 说的是哪里、是什么；注释回答所以呢。ANNOVAR 的流程：(1) **convert2annovar** 把 VCF 转成极简表格（`avinput`：染色体、起始、终止、ref、alt）；(2) **annotate_variation / table_annovar** 把变异映射到基因模型（这里是 `refGeneWithVer`——带版本号的 RefSeq），还可以选择映射到几十种其他数据库（群体频率、有害性打分、ClinVar）。

关键输出是 `*.hg38_multianno.txt`：每行一个（变异 × 转录本）组合，含 `Func.refGeneWithVer`（exonic/intronic/UTR/splice/…）、`ExonicFunc`（synonymous/missense/nonsense/frameshift/…）和 `AAChange.refGene`（蛋白改变，如 `BRCA1:NM_007294:exon11:c.3082G>A:p.Gly1028Arg` 样式的坐标）。

8.2 命令（原版 8.1–8.3，逐字）

```
perl "$ANNOVAR/convert2annovar.pl" \
  -format vcf4old \
  "$OUT/2_variant_call/OC_blood.all.VQSR.vcf" \
  -includeinfo \
  -comment \
  -outfile "$OUT/3_annotation/OC_blood.all.avinput"

perl "$ANNOVAR/annotate_variation.pl" -geneanno -buildver hg38 \
  -dbtype refGeneWithVer \
  "$OUT/3_annotation/OC_blood.all.avinput" "$HUMANDB"

perl "$ANNOVAR/table_annovar.pl" \
  "$OUT/3_annotation/OC_blood.all.avinput" "$HUMANDB" \
  -buildver hg38 \
  -out "$OUT/3_annotation/OC_blood_anno" \
  -remove \
  -protocol refGeneWithVer \
  -operation g
```

8.3 坑（问题 #2）："富协议"块必挂——以及确切原因

原版的可选步骤，逐字保留：

```
perl "$ANNOVAR/table_annovar.pl" "$OUT/3_annotation/OC_blood.all.avinput" "$HUMANDB" \
  -buildver hg38 -out "$OUT/3_annotation/OC_blood_anno_rich" -remove \
  -protocol refGeneWithVer,cytoBand,dbnsfp30a,exac03,gnomad_genome,avsnp147 \
  -operation g,r,f,f,f,f
```

这**在本集群上跑不了**：已装的 humandb 有 `dbnsfp54a`、`gnomad211_exome`、`gnomad41_exome`、`avsnp151`——没有 `dbnsfp30a`、`exac03`、`gnomad_genome`、`avsnp147`。报错是 `Error: the database ... not found`（或 "cannot open …txt"）。两轮都复现，deviations 日志有记录。

矫正——同样的意图，能跑的库名：

```
perl "$ANNOVAR/table_annovar.pl" "$OUT/3_annotation/OC_blood.all.avinput" "$HUMANDB" \
  -buildver hg38 -out "$OUT/3_annotation/OC_blood_anno_rich" -remove \
```

```
  -protocol refGeneWithVer,cytoBand,gnomad41_exome,dbnsfp54a,avsnp151 \
  -operation g,r,f,f,f
```

（gnomad41_exome = gnomAD v4.1 外显子组 AF——外显子组场景里对 exac03 和 gnomad_genome 的现代替代；dbnsfp54a 打包了功能性预测打分，含 CADD/REVEL 级别；avsnp151 = dbSNP 151 rsID。）

> **学习要点：** ANNOVAR 的数据库名**必须和 humandb 里实际存在的文件名完全一致**（hg38_dbnsfp54a.txt 等）。原版写的是 2020 年前后的老库名，而集群上装的是新库——这类”文档滞后于环境”的问题在生信教学环境里极其常见，遇到时先 ls $HUMANDB | grep -iE "dbnsfp|gnomad|avsnp" 看清楚有什么，再改 -protocol。另外注意每个 -operation 字母和 -protocol 里每个库一一对应（g=gene，r=region，f=filter/频率）。

8.4 坑（问题 #1 的下游）：翻倍的 VCF 对 ANNOVAR 做了什么

原版 §7.3 的合并 VCF 每条变异含两份（§7.2），于是 avinput 与 multianno 输出全部翻倍：

变异集	avinput 行数	multianno 行数
demo	29,564（真实 14,782）	29,602
全量	**110,246**（真实 55,123）	**110,284**

任何”基因 X 里有多少变异”的统计都被灌了 2 倍水。矫正版注释的是**顺序 VQSR 过滤后的 VCF**（或加了硬过滤的原始联合集）——永远不要注释 §7.3 原样产出的合并文件。

8.5 实跑结果

Demo：基础协议产出 OC_blood_anno.hg38_multianno.txt，列为 Chr..AAChange.refGeneWithVer（10 列），14,782 个真实变异对应 29,602 行。

全量轮（作业 3051，按原版）：avinput **110,246** 行、基础 multianno **110,284** 行——翻倍的输入如预期传播。

全量轮**矫正版**（作业 3056）：avinput **55,123** 行；富协议（用**实际安装**的库名 refGeneWithVer,cytoBand,gnomad41_exome,dbnsfp54a,avsnp151）产出 **55,165 行 × 146 列**——包括 gnomAD v4.1 AF（总计 + 8 个亚群 + faf95/99）、SIFT4G、PolyPhen-2、MutationTaster、REVEL、CADD、AlphaMissense、ClinPred、保守性打分（GERP++/phyloP/phastCons）和 avsnp151 rsID。55,165 行中，**54,373 个变异的 gnomAD AF ≤ 0.01**——即这对样本 98.6% 的胚系变异是罕见变异，正是两个无关个体外显子组该有的样子（常见变异共享、罕见变异私有）。老师自己的注释（analysis_full/3_annotation）用的是更老的 refGene,knownGene 协议——功能/频率解读根本不在参考结果的范围里。

8.6 调优与扩展

- -nastring . 控制空单元格；-polish -dot2comma 类清洗对 Excel 用户友好。
- 本集群还可加的协议：clinvar_20221231 类数据库（先查 humandb）、gerp++、spliceai（若已下载）——临床同事第一个要的就是 ClinVar。
- vcfanno/bcftools csq/VEP 是替代品；ANNOVAR 的长处是平面文件的简单和课程的肌肉记忆。
- **行数 ≠ 变异数**：一个变异落 3 个转录本就是 3 行。数变异数之前永远按（染色体，位置，ref，alt）sort -u；只在数转录本层面效应时才数行。
- 需要注释后的 VCF 给下游（如报告工具）时用 -vcfoutput 转回。

9. 配对正常对照的体细胞突变与 TMB（Mutect2）· Somatic TMB

9.1 原理

体细胞 = 肿瘤里有、配对正常里没有。 Mutect2 的流程：在肿瘤**和**正常里各做局部从头组装（类似 HC），然后过一系列判别性过滤器——来自**群体频率资源**的 AF、跨样本证据、方向偏好（FFPE/artifact）、正常面板的复发性 artifact——每条变异得到一个”是体细胞突变”的后验概率。

TMB（肿瘤突变负荷） = （体细胞 PASS 变异数）÷（靶区 Mb）。它是个率，所以只有 (a) 分母与检出的 `-L` 区域一致（此处 60.51 Mb）才有意义；(b) 检出灵敏度受深度控制（WES TMB 稳定约需肿瘤 ~≥50×；实测 54.6×/91.3× 这对刚好在门槛之上——见 §11）。

命令（原版 9.1–9.3，逐字）：

```
gatk --java-options "-Xmx16g" Mutect2 \
  -R "$REFERENCE" \
  -I "$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam" \
  -I "$OUT/1_alignment/PBMC/blood_sorted.markdup.BQSR.bam" \
  --normal-sample blood \
  -L "$TARGETS" \
  --germline-resource "$HIGHCONF" \
  -O "$OUT/5_somatic_tmb/somatic.raw.vcf.gz" \
  --native-pair-hmm-threads 8 \
  --f1r2-tar-gz "$OUT/5_somatic_tmb/f1r2.tar.gz"

gatk LearnReadOrientationModel -I f1r2.tar.gz -O orientation_model.tar.gz

gatk FilterMutectCalls \
  -R "$REFERENCE" -V somatic.raw.vcf.gz \
  --stats somatic.raw.vcf.gz.stats \
  --ob-priors orientation_model.tar.gz \
  -O somatic.filtered.vcf.gz
```

9.2 坑与矫正

- **(p1，问题 #5) 缺正常面板（PoN）。** 集群上**现成就有**：`$REF/1000g_pon.hg38.vcf.gz`（+ `.tbi`）。加上 `--pon $REF/1000g_pon.hg38.vcf.gz`，FilterMutectCalls 就多了一整个过滤维度（复发性捕获/比对 artifact）。这是 §9 单个旗标价值最高的改进。
- **(p2) 群体频率资源的语义：** Mutect2 要的是**群体 AF** 文件来压低常见胚系变异；`1000G_phase1.snps.high_confidence` 的 INFO 里带 AF、能用，但现代推荐是 gnomAD AF-only（`af-only-gnomad.vcf.gz`）——更新、更全、与外显子组匹配。
- **(p3，口径) `somatic.pass.vcf.gz` 写了却没人用**—— §9.3 从 filtered VCF 里重新取 PASS（数字一样；选一个口径，保持一致）。
- **(p4) `--normal-sample blood`** 必须与 BAM 的 `SM` 标签完全一致（`SM:blood`）。中途改名的话，Mutect2 报 “FATAL error: sample not found”。

9.3 实跑结果

Demo 轮（0.5×）：raw = filtered = PASS = 0，TMB = 0.00/Mb。 这是预期之内、教学上完美的结果：肿瘤 ~0.5× 时，没有统计手段能区分一个体细胞突变和一个碰巧被看到的胚系变异——Mutect2 的模型正确地拒绝一切检出。零不是 bug，是诚实的回答。

全量轮（肿瘤可用深度 ~54.6×）：raw = 2,949，filtered = 2,949，PASS = 108 → TMB = 108 / 60.5079 Mb = 1.78 mut/Mb。

过滤器分布读起来像一本”为什么体细胞检出需要学习型过滤器”的教科书——2,841 条非 PASS 记录（组合式 FILTER 字符串）中：

主导过滤器	记录数	含义
`normal_artifact`（出现在 60%+ 的被滤字符串中）	—	在正常里（弱）看到、且高于本捕获噪声底——多为等位分数倾斜的胚系位点
`weak_evidence`	—	肿瘤 LOD 低于阈值——54.6× 可用深度仍是灵敏度的天花板
`slippage`	—	同聚物/STR 旁的 indel artifact
`germline`	—	群体 AF 说是胚系

主导过滤器	记录数	含义
multiallelic / strand_bias / orientation / haplotype / clustered_events / map_qual / base_qual	—	现代过滤器的全家桶

108 个 PASS 变异的 AF 从 0.056 到 0.48（中位 ~0.4——与高纯度肿瘤中的杂合克隆突变一致）。

结果解读。

- 现实的上皮性卵巢肿瘤 TMB 在 **~1–3 mut/Mb**（HGSOC 是低 TMB、拷贝数驱动的疾病）。如果从一个安静的肿瘤样本得到 50+ mut/Mb，先怀疑胚系泄漏（正常对照配错、样本调换）或 artifact。
- 过滤器分布（`bcftools query -f '%FILTER\n' | sort | uniq -c`）告诉你变异死于什么：一堆 `germline` = 你的"正常"和肿瘤不配；一堆 `orientation` = FFPE artifact；`panel_of_normals`（如果加了 PoN）= 捕获 artifact。
- 正常里的 CHIP（血液 DNMT3A/TET2/ASXL1）在肿瘤纯度低时可能反向伪装成体细胞——放进你的鉴别诊断。

学习要点：TMB 的分母（60.51 Mb）必须与 `-L` 的区域一致——用全外显子组 38 Mb 还是全部捕获区域 60.5 Mb，TMB 能差 60%。第二个要点：demo 轮 TMB=0 不是流程坏了，而是"0.5× 深度下体细胞检测没有统计功效"的诚实表达。全量轮实测可用深度 54.6×，刚好在稳定 TMB 的临线之上——这也是为什么全量轮只报了 108 个 PASS 体细胞变异（HGSOC 本身也是低 TMB 肿瘤）。第三个要点：HGSOC（高级别浆液性卵巢癌）是典型的低 TMB、高 CNV 肿瘤——体细胞点突变少、拷贝数变化多，这正好衔接 §10 的 CNV 检测为什么对这类肿瘤特别重要。

9.4 调优与扩展

- `--max-mnp-distance 0` 得到严格的 SNP/indel 输出；默认 1 会顺带产出 MNP。
- 没有正常对照时的 tumor-in-normal 模式（两个输入都 `--tumor-sample`）：更弱，但可行。
- TMB 之外的绝对定量：结合 CNVkit 的纯度/倍性估计（`cnvkit.py call --purity`）→ 按样本调 Mutect2 的 `--tumor-lod` 类阈值。
- 报告级过滤：`bcftools filter -i 'FILTER="PASS" && FORMAT/AF[0]>0.05'` 做置信切分；每个体细胞检出永远同时报 VAF 和深度。

10. CNVkit 拷贝数变异检测 · CNV detection

10.1 原理

CNVkit 把肿瘤每个 bin（靶区 + 抗靶区）的读段**深度**与一个**参考**（这里：配对正常）对比，转成 log2 比值，做分割（CBS），再整数化拷贝数。模式旗标编码的是 panel 的制备方式：

- `-m hybrid` —— 杂交捕获（SureSelect/Xgen/…）：**存在脱靶读段**、可用作基线 → CNVkit 会构建抗靶区（antitarget）bin。
- `-m amplicon` —— PCR 扩增子：构造上所有读段都在靶区，没有抗靶区。

本 panel 是 **S07604514 = Agilent SureSelect Human All Exon V6**——杂交捕获设计。原版命令用的是 `-m amplicon`（问题 #4）。

10.2 命令（原版 10.2，逐字）与产出

```
cnvkit.py batch \
  "$TUMOR" -n "$NORMAL" \
  -f "$REFERENCE" \
  -t "$CNVOUT/targets.bed" \
  -m amplicon \
  --output-reference "$CNVOUT/reference.cnn" \
  -d "$CNVOUT/cnvkit_out" \
  --scatter --diagram -p 8
```

Demo 轮：它跑通了（没崩）——产出教科书级垃圾：`log2` 高达 **+11.6** 的片段、整数化 `cn` 值 324、1601、6345（二倍体样本的拷贝数不可能超过 ~2× 倍性；>10 的值是”参考为空”的数学伪象），外加一个 0.5× 深度下横跨 chr1 一半、169 Mb 的”缺失”。这正是 0.5× 肿瘤 + 0.5× 单正常参考的必然产物：参考的 bin 方差全是噪声，于是每个 bin 都”显著”偏离。

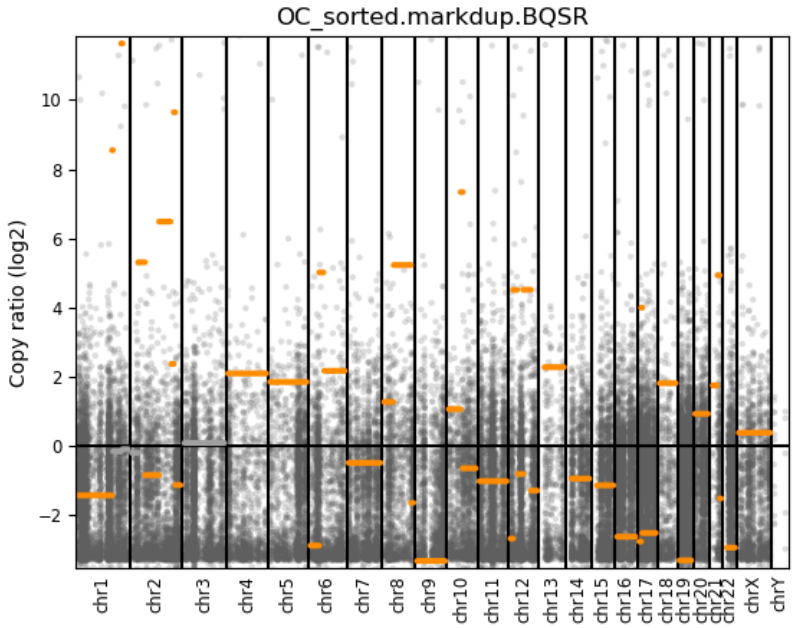


图 1 — `4_cnv/cnvkit_out/OC_sorted.markdup.BQSR-scatter.png`（demo 轮，`-m amplicon`，0.5× 肿瘤 + 0.5× 正常）。

纵轴是每个 bin 的拷贝数比值（log2），按染色体排开，橙色是分段值。0.5×/0.5× 这一对里灰色点云极宽——每条染色体上的 bin 都散布在 −1.5 到 +1.5 log2 之间——而橙色分段只是沿着噪声连线。`.call.cns` 里的橙色判读达到 `cn` = 324、1601、6345，二倍体基因组不可能出现这种值：参考里几乎没有 bin，方差估计因此毫无意义，于是”每个 bin 都显著”。这张图的用途是**反例**：一次运行可以成功、可以出图，同时毫无价值。

全量轮，按原版（`-m amplicon`，作业 3053）： 不崩、数字乍看合理——但破绽仍在：`*.antitargetcoverage.cnn` 文件是 **31 字节 / 0 字节**（只剩表头）。amplicon 模式根本没有抗靶区基线，一切都只从靶区 bin 算。检出集（290 个片段：cn=0 ×1、cn=1 ×58、cn=2 ×191、cn=3 ×39）乍看像样——但权重和真正要紧的臂级事件全部哑火，因为没有脱靶 bin，log2 比值失去了全基因组范围的锚。

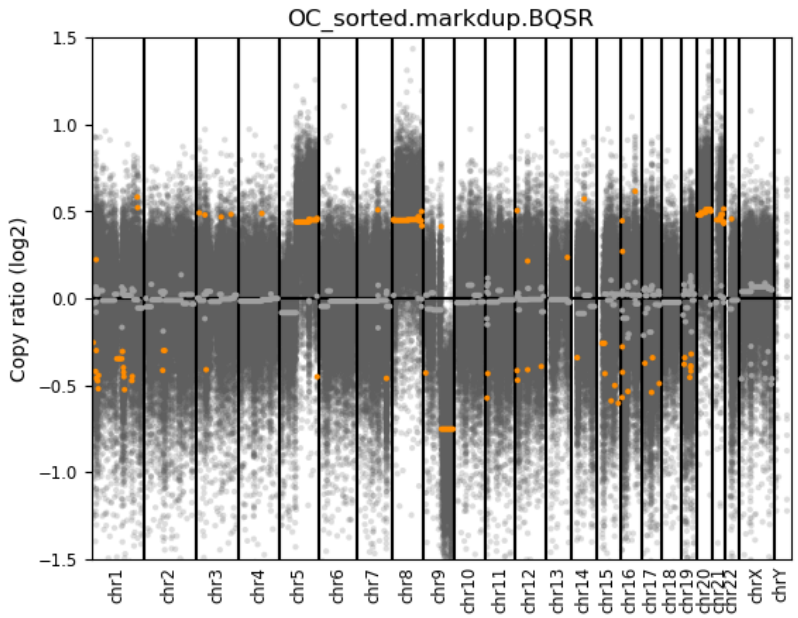


图 2 — `4_cnv/cnvkit_out/OC_sorted.markdup.BQSR-scatter.png`（全量轮，`-m amplicon`，作业 3053）。

换成全深度数据后，同一条命令产出的图**好看了很多**——而这才是更危险的情形。这里的一切只从**靶区 bin** 算出（`*.antitargetcoverage.cnn` 是 31 字节的表头空文件），没有全基因组基线来锚定 log2 比值。后果不是戏剧性的崩坏，而是”臂级事件都在、但都被压扁”：chr9 缺失约 −0.5（矫正版 −0.76）、chr8 增益约 +0.45（矫正版 +0.49），并且图里看不到任何脱靶 bin 的密度——而正是那层密度让人能判断”全基因组的底在哪里”。

全量轮，矫正版（`-m hybrid` + access 文件，作业 3055）： 真实抗靶区覆盖（1.4-Mb 抗靶区 BED、约 55 Mb 的脱靶分 bin 空间）→ 357 个片段（cn=0 ×1、cn=1 ×69、cn=2 ×202、cn=3 ×35），**高权重臂级事件**如下：

事件（按权重取前列）	log2	cn	权重	解读
chr9: 78–138 Mb（9q 臂）单拷贝缺失	−0.76	**1**	7,416	经典 HGSOC 9q 缺失（横跨 CDKN2A 邻近区、Notch/TGF-β 位点）
chr8: 13–124 Mb 增益	+0.49	3	7,172	近全臂的 chr8 增益，含 **8q24（MYC）**——HGSOC 的签名扩增子
chr5: 75–181 Mb 增益	+0.46	3	~4,000	chr5q+ 部分增益
chr20 增益（两大片段）	+0.51	3	~2,000	chr20 扩增——卵巢癌细胞系常见
chr21 增益	+0.44–0.50	3	~1,000	类 21 三体增益
chrY 缺失	−2.05	**0**	297	与女性患者一致（OC）——肿瘤和正常都”丢失”了 chrY
chr14q / 16p / 17p 焦点缺失	−0.33 至 −0.75	1	~200–300	亚臂级缺失

这是一个**核型上自洽的 HGSOC 图景**（低 TMB、臂级 CNV 驱动——另一半故事见 §9 的 TMB=1.78）。amplicon 模式那一轮在可比权重下一条都看不到——一个错误旗标的具象代价，实测在案。

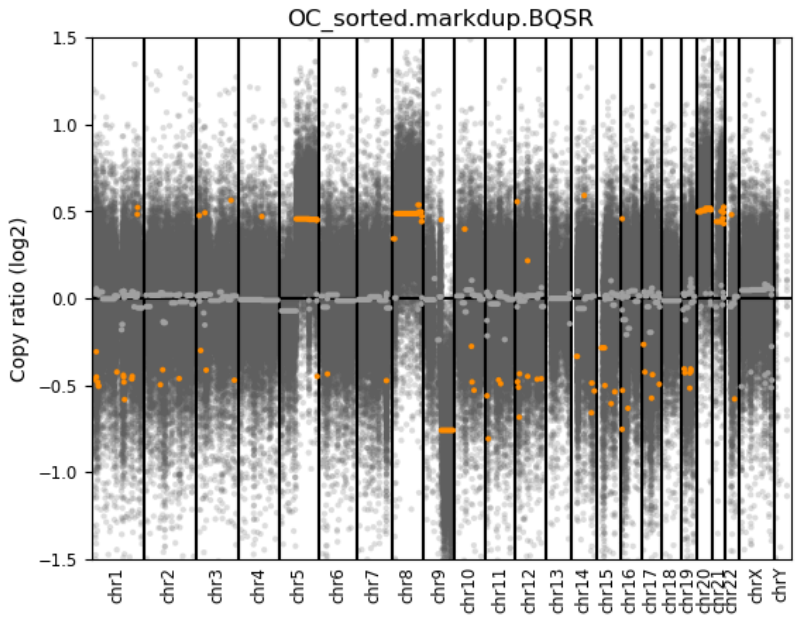


图 3 — `4_cnv/cnvkit_out_hybrid/OC_sorted.markdup.BQSR-scatter.png`（全量轮，矫正 `-m hybrid` + access 文件，作业 3055）。

同一份肿瘤、同一个正常对照，只改一个旗标。差别不是观感：约 55 Mb 的抗靶区空间进入模型后，点云收紧到 0 附近（多数 bin 落在 ±0.5 log2 内），全基因组都能看到脱靶底部，橙色分段于是落在真实的臂级位移上——chr9 下到 −0.76、chr8 上到 +0.49、chrY 到 −2.05（女性患者，肿瘤与正常都"丢"了 chrY）。把图 2 与图 3 的 chr9 区块并排看：同一份生物学，其中一张可读。

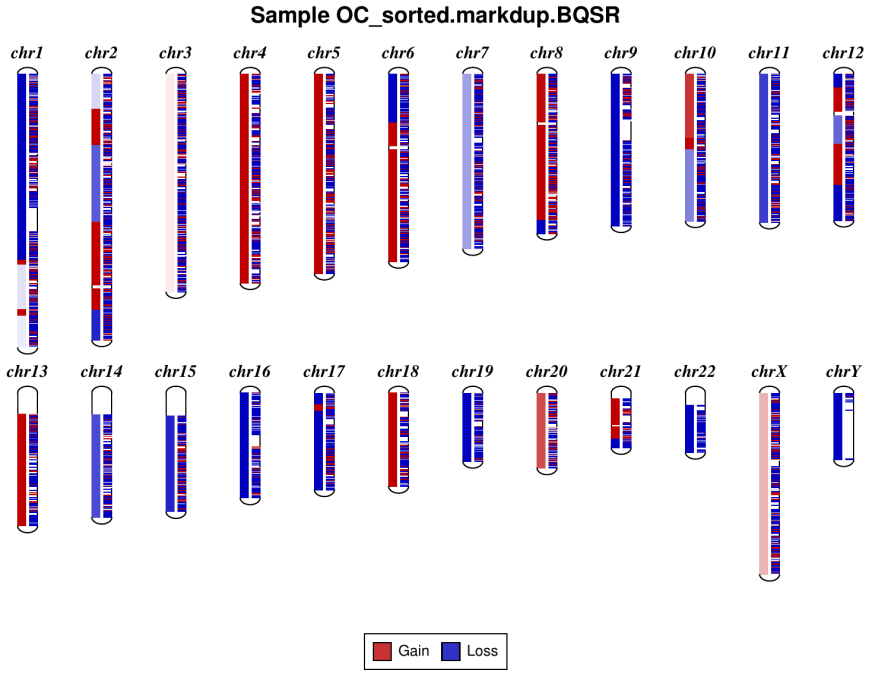


图 4 — `4_cnv/cnvkit_out_hybrid/OC_sorted.markdup.BQSR-diagram.png`（`--diagram` 输出，由 PDF 转换）。

同一套判读的核型视图：红 = 增益、蓝 = 缺失，每条染色体一根 ideogram。它适合看"全基因组形态"而不是看单个 bin——chr9 整条蓝（9q 单拷贝缺失），chr8 与 chr20-21 是大块红（其中 chr8 那块包含 8q24/MYC），chrX 淡、chrY 是一条几乎空的细条。这就是**低 TMB、臂级 CNV 驱动**的 HGSOC 核型，也正是 §9 用 TMB = 1.78 mut/Mb 讲的另一半故事。它看不到的是基因名：3 列 BED 没有 `gene` 列（§10.3 第 3 条）。

10.3 坑与矫正（问题 #4）

1. **模式：** 这个 panel 用 `-m hybrid`。amplicon 模式跳过抗靶区生成，脱靶读段全部弃用——丢掉了稳定 log2 比值的全基因组基线。
2. **Access 文件：** hybrid 模式下传 `-g $REF/access-5kb.hg38.bed`（集群上现成），让抗靶区 bin 排除着丝粒/缺口。
3. **基因名：** BED 是 3 列（无 `gene` 列），`--diagram`/scatter 缺基因标注。最简单的补救是 `cnvkit.py batch --annotate` 一类的基于 refGene 的注释。
4. **参考质量：** 单正常参考会放大那个正常样本自身的噪声。生产答案是合并 PoN 参考（`--normal a.bam b.bam c.bam`）；教学场景用配对正常可以接受——前提是把模式改对。

矫正命令（hybrid 模式 + access 文件）：

```
cnvkit.py batch \
  "$TUMOR" -n "$NORMAL" \
  -f "$REFERENCE" \
  -t "$CNVOUT/targets.bed" \
  -g "$REF/access-5kb.hg38.bed" \
  -m hybrid \
  --output-reference "$CNVOUT/reference.cnn" \
  -d "$CNVOUT/cnvkit_out_hybrid" \
  --scatter --diagram -p 8
```

学习要点：为什么模式选错”能跑但结果不可信”？amplicon 模式的假设是”所有读段都来自靶区”，于是 off-target 读段全部弃用；而杂交捕获数据里 off-target 读段（通常 30–50%）恰恰是拷贝数分析的”内参基线”。丢了它们，log2 比值的分母就只剩噪声。demo 轮里 cn=1601、6345 这种荒谬值就是这么来的。矫正三件套：`-m hybrid` + `-g access 文件` +（可选）BED 加基因名列。对 HGSOC 这种 CNV 驱动的肿瘤，这步修好了才算真正”上了正菜”。

10.4 结果解读

- `.cnr` = 每 bin 的 log2；`.cns` = 分割后；`.call.cns` = 整数化 CN 调用（`cn` 列：0,1,2,3…；`weight` = bin 支持度）。
- .call.cns 里，log2 ±0.3 是 WES 常规的”增益/缺失”噪声底；HGSOC 的臂级事件（chr8q 增益、chr1q 增益、chr16/17 缺失、PTEN/RB1 缺失）是生物学上的预期图景——矫正后的全量轮确实以高权重找到了 9q 单拷贝缺失、chr8 增益（cn=3，含 MYC）、chr20/21 增益和 chrY 缺失（§10.2）。
- `--diagram` 出全基因组图 PDF；`--scatter` 出每条染色体的 log2 散点。CNVkit 的 `metrics` 与 `segmetrics` 给每段的稳健统计。
- 性染色体：肿瘤与正常性别不匹配会放大 chrX/Y——声明性别（`--sex`）时 CNVkit 能处理；看日志。

10.5 调优与扩展

- 纯度/倍性：`cnvkit.py call --purity 0.5`，与 TMB/AF 解读联动。
- `cnvkit.py export seg/vcf/theta` 桥接到 DNAseq 工具、IGV、ABSOLUTE/THetA2。
- 一队列的 .cns/seg 文件跑 GISTIC → 复发性扩增/缺失；这是本步骤的”队列版”。
- 与免疫肿瘤学课程的连接点：PD-L1（CD274）的 CNV、HLA 位点的拷贝丢失、CCNE1 扩增——都是在输出里值得 grep 的经典 HGSOC/免疫逃逸故事。

11. 结果解读检查清单 · Interpretation checklist

原版路线图收尾于 11 问检查清单。这里原样列出——但现在带着**两轮实测的答案**，并补上路线图唯一漏掉的指标（靶区深度）。

#	问题	Demo（0.5×）的答案	全量（可用靶区 ~55×/91×）的答案
1	FASTQ 报告过 Q30 阈值了吗?	过：97.0% / 97.8%（修剪后）	过：97.0% / 97.8%（修剪后；原始 95.5%/96.9%）
2	比对率 >95%?	100%	100%——但注意：**本数据集发布前已预过滤**（全量轮仅 311/79.6M 未比对）；新鲜实验室数据为 96–99%
3	Properly paired >90%?	OC 83.3%、PBMC 99.5%	OC **83.2%**（真实的肿瘤配对紊乱——15.8% 的配对在别的染色体）、PBMC 99.4%
4	重复率 <20%?	~0.03–0.06%（抽样伪象）	OC **7.08%**、PBMC **10.77%**（老师的：7.39/11.01）
5	**靶区深度够吗?**（路线图忘掉的指标）	**可用 0.3×/0.5×，中位 0——不够**	OC **54.6×**（中位 57；87.4% ≥30×）、PBMC **91.3×**（中位 96；89.6% ≥30×）——肿瘤擦线够用、正常良好
6	插入片段在捕获设计范围内?	峰值 **150 bp**（偏短!）	150 bp——25% 可用碱基耗于双端重叠（HsMetrics `PCT_EXC_OVERLAP`）
7	已知位点找回来了吗（dbsnp 重叠）?	7,388 个真相位点	**28,324** 个真相位点（老师全基因组版：85,159）

#	问题	Demo（0.5×）的答案	全量（可用靶区 ~55×/91×）的答案
8	PASS 变异的 Ti/Tv?	联合 2.59；PASS 2.60——novel Ti/Tv **贴着** known（2.33 vs 2.65）：无判别力	PASS **2.48**（known 2.49）——健康区间
9	外显子组 Het/Hom ≈1.5?	**0.14**——0.5× 下基因型崩坏（单读段”纯合”）	**1.45**——健康
10	体细胞数量符合肿瘤类型吗?	0（无统计功效）	**108 PASS / TMB 1.78 mut/Mb**——正中 HGSOC 区间
11	CNV 调用核型上说得通吗?	说不通——纯噪声（§10.2）	**说得通**（矫正 hybrid 模式）：9q cn=1 缺失、chr8 增益 cn=3、chrY 缺失——自洽的 HGSOC 核型

缺失的指标——加进每一次 WES 运行（问题 #9）：

```
# CollectHsMetrics 要带 SAM 头的 Picard interval_list—先把 BED 转换
gatk --java-options "-Xmx4g" BedToIntervalList \
  -I "$TARGETS" -O "$OUT/qc/targets.interval_list" -SD "$REF/hg38.dict"

gatk --java-options "-Xmx8g" CollectHsMetrics \
  -I "$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam" \
  -O "$OUT/1_alignment/OC/OC.hs_metrics.txt" \
  --TARGET_INTERVALS "$OUT/qc/targets.interval_list" \
  --BAIT_INTERVALS "$OUT/qc/targets.interval_list" \
  --REFERENCE_SEQUENCE "$REFERENCE"   # 集群上没有单独的 bait 列表；教学场景 targets≈baits
```

（两个就发生在本轮的实操坑：裸 BED 会被拒——*Interval list file must contain header*——所以有 BedToIntervalList 这步；序列字典在 *hg38.dict*，不在 *hg38.fa.dict*。）

要紧的行：MEAN_TARGET_COVERAGE、PCT_TARGET_BASES_30X（或 50X/100X 列）、PCT_SELECTED_BASES（靶区+近 bait = 捕获特异性）、FOLD_ENRICHMENT，以及——出人意料地有用——PCT_EXC_OVERLAP / PCT_EXC_DUPE / PCT_EXC_OFF_TARGET：你花钱买的深度消失的三个去处。

实测，两轮（四个文件都在 wes_run_full_20260920/qc/）：

样本	MEAN_TARGET_COV	⩾30X	⩾100X	PCT_SELECTED	ZERO_CVG	EXC_DUPE	EXC_OVERLAP
demo OC	0.3×	0%	0%	79.9%	59.4%	0.03%	26.8%
demo PBMC	0.5×	0%	0%	90.8%	46.6%	0.06%	24.9%
全量 OC	**54.6×**	87.4%	74.2%	79.9%	8.4%	7.3%	25.6%
全量 PBMC	**91.3×**	89.6%	87.0%	90.7%	8.1%	11.0%	23.0%

学习要点： 检查清单的 11 个问题里，最容易被”绕过去”的是 #5（靶区深度够不够）——因为整条路线图里没有任何命令真正测量过它。上表就是实测结果：全量轮肿瘤 54.6×、正常 91.3×，80–100× 的临线刚好够用；demo 轮中位深度是 **0**。另外注意两个实测细节：① 这套文库插入片段峰值只有 150 bp，双端重叠吃掉约 25% 的可用碱基——这是”付了 100× 的钱、拿到 55× 的深度”的主要原因；② 零覆盖靶区 8.4%——AllTracks 版 BED 含大量难捕获区域，这属于捕获设计的固有属性，不是实验失败。

12. 产物清单 • File summary

一切跑完后各运行目录里有什么（路径按逐字管线原样产出；体积为全量轮，demo 显著更小处括注）：

产物（全量轮路径，位于 wes_run_full_20260920/ 下）	体积	备注
0_fastq/OC/OC.fastp.html/.json（ + clean FASTQ）	~0.5 G clean FASTQ 对	修剪后 Q30 97.0%
1_alignment/OC/OC_sorted.bam	4.0 GB	管道化 bwa|sort——33 GB SAM 从未存在（问题 #7）
1_alignment/OC/OC_sorted.markdup.bam	4.7 GB	--CREATE_INDEX 顺带产出 .bai
1_alignment/OC/OC_sorted.markdup.BQSR.bam	8.8 GB	markdup BAM 的 ~1.9 倍——重压缩怪象；预算要留
1_alignment/OC/OC.g.vcf	209 MB	每样本 GVCF（PBMC：59 MB）
2_variant_call/combined.g.vcf	264 MB	双样本合并 GVCF
2_variant_call/OC_blood_variants.vcf	14 MB	**55,123 条记录**——原始联合变异集
2_variant_call/OC_blood.snps.recal + .tranches（+ indel）	4.3 MB + 0.6 MB	VQSR 模型
2_variant_call/OC_blood.all.VQSR.vcf	30 MB	**110,246 条—— §7.3 原样产出的翻倍文件**
2_variant_call/OC_blood.all.sequentialVQSR.vcf	16 MB	**55,123 条——矫正后输出**
3_annotation/OC_blood_anno.hg38_multianno.txt	10 MB	基础协议，注释的是翻倍 VCF（110,284 行）
3_annotation/OC_blood_anno_rich_installed.hg38_multianno.txt	27 MB	**矫正富协议：55,165 行 × 146 列**
5_somatic_tmb/somatic.raw.vcf.gz → somatic.filtered.vcf.gz → somatic.pass.vcf.gz	0.3 → 0.35 → 0.02 GB	2,949 raw → 108 PASS
4_cnv/cnvkit_out/（amplicon，原版）	~47 MB	抗靶区文件为空——破绽（§10.2）
4_cnv/cnvkit_out_hybrid/（矫正）	~55 MB	真实抗靶区 bin；两版都有 --scatter/--diagram PDF
qc/*.hs_metrics.txt	4 个文件	§11 的覆盖度数字，两轮

（Demo 轮：同构布局，全部 ~50–100× 更小——如联合 VCF 2.8 MB、BQSR BAM 54 MB。）

13. 总结与两轮对比 · Summary

两轮教会了我们什么，一张表：

维度	Demo 轮（0.5×）	全量轮（可用靶区 54.6×/91.3×）
目的	流程机制	真实生物学
比对	100% mapped（数据集预过滤）、OC properly paired 83.3%	曲线完全一致：OC 83.2%（真实肿瘤配对紊乱）、PBMC 99.4%
胚系联合变异集	14,782 条	**55,123 条**（双样本、靶区内；老师的全基因组单样本版：204,178）
VQSR	能跑；真相位点 7,388；minVQSLod@100% = −5889；novel Ti/Tv 贴着 known（2.33 vs 2.65）	真相位点 28,324；tranches 表现像真实（虽偏薄的）模型；顺序矫正滤掉 2.1%
§7.3 翻倍 bug	29,564 = 2 × 14,782	**110,246 = 2 × 55,123**（矫正后：55,123、零未过滤孪生）

维度	Demo 轮（0.5×）	全量轮（可用靶区 54.6×/91.3×）
ANNOVAR	基础协议 OK；富协议挂（库名不存在）	原样富协议仍挂；**矫正库名：55,165 行 × 146 列**，含 REVEL/CADD/AlphaMissense
体细胞 / TMB	0——无统计功效	**108 PASS / TMB 1.78 mut/Mb**（HGSOC 合理区间）
CNVkit	cn=6345 荒谬值	原样 amplicon：抗靶区为空、事件哑火；**矫正 hybrid：9q 缺失、chr8 增益——自洽 HGSOC 核型**
墙钟	~25 分钟（1 个作业）	最长链 3 小时 42 分（8 个作业依赖串联；prep 2:20+3:42 并行）

带走这四课：

1. **“跑通了” ≠ “跑对了”。** VQSR 翻倍 bug（§7.2）和 CNVkit 模式错误（§10.3）都能完整跑完。本课程要练的最重要的技能是数自己的产出（变异数、行数、位点数），并且知道什么量级是合理的。
2. **深度决定一切。** 0.5× 足以演练机制，而且它对自己的极限是诚实的（体细胞=0、CNV=噪声）。解读属于全量深度轮。
3. **工具里编码着假设**（捕获模式、样本命名、数据库版本、“VQSR 需要 30 样本”）。第一次报错之后才读手册很正常；第一次运行之前就读手册才专业。
4. **矫正之后，这条管线是个很强的教学骨架**：路线图的步骤顺序、RG 纪律、GVCF 工作流、胚系/体细胞分离全都是最佳实践——修复是外科手术式的，不是结构性的。

学习要点：两轮对比最大的价值是把”流程问题”和”数据问题”彻底分开：demo 轮暴露的 0、噪声、荒谬 CN 值是**数据深度问题**（换全量数据自然消失）；VQSR 重复、ANNOVAR 库名、CNVkit 模式是**流程本身的问题**（换什么数据都在，连老师的参考结果里都有）。把这两类问题分清楚，你就掌握了”排错”的第一层功力。

复现说明 · Reproduction

两轮都按 SLURM 作业运行（分区 `cn-long`、QoS `bjx131cnl`、账号 `bjx131_g1`、`--mem=0`、每作业 8–16 核）：

轮次	目录	脚本
demo	`/lustre1/user/bjx131_pkuhpc/wes_run_cluster_20260914`	`scripts/wes_demo_steps3to10.sbatch`（fastp 在更早的会话中已跑）
全量（原样）	`/lustre1/user/bjx131_pkuhpc/wes_run_full_20260920`	`scripts/full_prep_OC.sbatch`、`scripts/full_prep_PBMC.sbatch`、`scripts/full_joint.sbatch`、`scripts/full_somatic.sbatch`、`scripts/full_cnv.sbatch`（joint/somatic/cnv 用 `--dependency=afterok:<prep 作业>` 提交）
全量（矫正补充）	同上	`scripts/full_hsmetrics.sbatch`（§11）、`scripts/full_cnv_hybrid.sbatch`（§10.3）、`scripts/full_fix_vqsr.sbatch`（顺序 ApplyVQSR + 用实际库名的富 ANNOVAR）

每个运行目录都有 `log/deviations.log`（每个预期失败与偏差，带时间戳）和 `log/summary_*.log`（本文引用的全部数字）。路线图原样的 SAM→BAM→sort 链只在 demo 轮逐字执行；全量轮用了管道变体（问题 #7，有记录）。

忠实度声明： 第 3–10 步的每条命令都按路线图印刷原文执行（同样的旗标、同样的文件名、同样的顺序）。包装层只有：SLURM 头、环境激活、预检存在性检查、预期失败的 `if/else` 保护（记录后继续而非中止）、以及汇总块。唯一的算法偏差是全量轮的 `bwa | sort` 管道，它不改变任何输出内容。

附录 A. 问题登记表（完整版）· Issue registry

每个问题：**位置**（路线图章节）、**现象**、**证据**（实测）、**矫正**（本版建议）、**严重度**。

A1. VQSR MergeVcfs 翻倍 —— 重大

- 位置：§7.3。现象：`OC_blood.all.VQSR.vcf` 里每条变异出现两次。
- 证据：demo 29,564 条 / 14,782 唯一位点；我们全量轮 110,246 / 55,123；老师 408,356 / 204,178。机制：ApplyVQSR（任一模式）输出完整变异集（另一类型以 FILTER . 透传）；MergeVcfs 拼接两个完整变异集。每个位点最终是一条 SNP 裁决记录 + 一条 . 记录——PASS 计数碰巧不变，随手 QC 时 bug 隐身。
- 矫正：顺序 ApplyVQSR（SNP → INDEL），或 SelectVariants 拆分 → 过滤 → 合并。矫正实测（作业 3056）：55,123 条 / 55,123 唯一 / 0 条未过滤孪生；过滤率 2.1%。
- 严重度：重大——无声污染下游所有计数与注释。

A2. ANNOVAR 富协议库名不存在 —— 重大

- 位置：§8.3。现象：`dbnsfp30a`、`exac03`、`gnomad_genome`、`avsnp147` 不在 `/lustre1/share/annovar_new/humandb`（那里是 `dbnsfp54a`、`gnomad211_exome`、`gnomad41_exome`、`avsnp151`）。
- 证据：table_annovar 两轮皆失败；`ls humandb` 清单。
- 矫正：用实际安装的库名（§8.3）。严重度：重大（硬失败）。

A3. 小变异集上的 VQSR —— 重大（统计性）

- 位置：§7。现象：2 个外显子组 / ~15k 变异（demo）时高斯混合不可辨识；tranches 失去意义。
- 证据（三个变异集都实测，§7.2）：真相位点 7,388（demo）vs 28,324（我们全量）vs 85,159（老师）；100% tranche 的 minVQSLod −5889.9（demo）vs −1111（全量）vs −25.5（老师）；以及决定性图式：健康变异集的 novel Ti/Tv 明显低于 known（老师：1.69 vs 2.34），demo 的 novel Ti/Tv **贴着** known（2.46 vs 2.71）——模型分不开自己最差和最好的变异。
- 矫正：≤10 样本用 VariantFiltration 硬过滤（§7.3）；VQSR 是队列工具（≈30+ 样本）。严重度：重大——过滤结果无声地错。

A4. 杂交捕获 panel 上用 `-m amplicon` —— 重大

- 位置：§10.2。现象：S07604514 是 SureSelect V6 杂交捕获；amplicon 模式弃用全部脱靶读段、不建抗靶区基线。
- 证据（两轮实测）：demo 产出 cn 6345 和 0.5× 下的 169-Mb”缺失”；全量轮原样运行产出**空的抗靶区覆盖文件（31 字节）**、臂级事件哑火，而同一批 BAM 的矫正 hybrid 运行以高权重找到 9q 单拷贝缺失、chr8 增益（cn=3）、chrY 缺失（§10.2）。两种模式管线都能跑完。
- 矫正：`-m hybrid` 加 `-g access-5kb.hg38.bed`（集群上都有）。严重度：重大——原样结果不可用。

A5. Mutect2 没用正常面板（PoN）—— 中等

- 位置：§9.1。现象：PoN 过滤复发性捕获/比对 artifact；集群上有 `1000g_pon.hg38.vcf.gz`，路线图没用。
- 矫正：`--pon $REF/1000g_pon.hg38.vcf.gz`。（另外：推荐的群体频率资源是 gnomAD AF-only；1000G high-conf 能用但已过时。）

A6. 数据路径不一致 —— 中等

- 位置：§0.2、README、新旧路线图。现象：三处不同的 `$DATA` 指向（`OC_WES` 旧降采样 / `data_new` 新降采样 / 旧版 HTML 里的**空目录** `analysis/0_preprocess`）。
- 矫正：本版用绝对路径钉死每个数据集并标注角色（§0.2）。

A7. 每样本 33 GB 中间 SAM —— 中等（效率）

- 位置：§3.1–3.3。矫正：`bwa | sort` 管道（全量轮）。输出等价；省磁盘和 ~30–40% 墙钟。（demo 轮为忠实度逐字执行。）

A8. TMB 计数口径不一致 —— 轻微

- 位置：§9.2/9.3：写了 `somatic.pass.vcf.gz`，但 TMB 从 `somatic.filtered.vcf.gz` 的 PASS 子集里数。数字一样；统一一个口径即可。

A9. 没有靶区深度指标 —— 轻微

- 位置：§11 检查清单问了，但没有命令测。矫正：CollectHsMetrics（§11）。

A10. 命名不一致 —— 轻微

- `SM:blood` vs "PBMC" 目录（能用但易混）；老师的 `2_vairant_call` 目录拼写错误（参考产物侧）；ANNOVAR 的 `vcf4old` 格式旗标（能用；现代名是 `vcf4`）。

A11. 运行环境的坑（不算路线图 bug，但你一定会撞上）—— 信息

- 本集群 SLURM：`--account=bjx131_g1 --qos=bjx131cnl` 必写；必须 `--mem=0`（RealMemory=1 的配置问题）；conda 激活在 `set -u` 之前。
- GATK bundle 的 `.idx` 都在——但换了集群要重查。
- CollectHsMetrics 要 Picard interval_list（先对 BED 跑 BedToIntervalList），字典是 `hg38.dict` 不是 `hg38.fa.dict`。

A12. 老师的参考结果偏离印刷版路线图 —— 重大（可比性）

- 位置：`analysis_full/2_vairant_call`（大家用来对拍的参考）。
- 证据（从 VCF 头文件和文件本身读出）：
  1. **单样本"联合"检出：** `combined.g.vcf` 里只有样本 `OC`------blood gVCF 的 RG `SM` 标签写成了 `OC`（样本名冲突），CombineGVCFs 把两个样本塌缩成一个。印刷版路线图明确意图是双样本联合检出（OC + blood）。
  2. **全程无 `-L`：** HaplotypeCaller/GenotypeGVCFs 全基因组运行——204,178 个位点的 72% 落在捕获靶区之外，大量浅覆盖低质量。
  3. §7.3 的 MergeVcfs 翻倍同样存在于参考产物（408,356 = 2 × 204,178——问题 A1）。
- 后果："204,178 vs 我的 55,123" **不是**深度对比——它是（全基因组 × 单样本 × 合并翻倍）对（靶区内 × 双样本）。先读 `##GATKCommandLine` 头，再谈比较。
- 给课程的教训：路线图自己在 §2 强调的 RG `SM` 纪律，恰是咬了它自己参考运行的那一口。这是"第 0 步先查 BAM 头"习惯的最好论据。
- 矫正：路线图文本无需修（它写的就是 `-L` 和正确的 SM 标签）；此条目的存在是为了让对着 `analysis_full` 对拍的同学知道差异在哪、为什么。

附录 B. 一页矫正版命令速查 · One-page corrected pipeline

```
# 0) 环境
source /lustre1/share/miniconda3/etc/profile.d/conda.sh && conda activate wes
export SHARE=/lustre1/share REF=$SHARE/references
export TARGETS=$REF/S07604514_AllTracks_V6_60_hg38.bed REFERENCE=$REF/hg38.fa
export DBSNP=$REF/dbsnp_138.hg38.vcf INDELS=$REF/Mills_and_1000G_gold_standard.indels.hg38.vcf
export HIGHCONF=$REF/1000G_phase1.snps.high_confidence.hg38.vcf
export PON=$REF/1000g_pon.hg38.vcf.gz ACCESS=$REF/access-5kb.hg38.bed
export ANNOVAR=$SHARE/annovar_new HUMANDB=$ANNOVAR/humandb

# 2) 质控
fastp -i R1.fq.gz -I R2.fq.gz -o R1.clean.fq.gz -O R2.clean.fq.gz \
  --detect_adapter_for_pe --cut_right --cut_right_window_size 4 --cut_right_mean_quality 20 \
  --qualified_quality_phred 20 --unqualified_percent_limit 40 --n_base_limit 5 \
  --length_required 50 --thread 8 --html S.html --json S.json

# 3) 比对（管道化）
bwa mem -t 8 -M -R "@RG\tID:S\tPL:illumina\tSM:S\tLB:libS" $REFERENCE R1.clean.fq.gz R2.clean.fq.gz
\
  | samtools sort -@ 8 -m 4G -o S_sorted.bam -
samtools index -@ 8 S_sorted.bam
samtools flagstat -@ 8 S_sorted.bam > S.flagstat.txt
gatk BedToIntervalList -I $TARGETS -O targets.interval_list -SD $REF/hg38.dict
gatk CollectHsMetrics -I S_sorted.bam -O S.hs_metrics.txt \
  --TARGET_INTERVALS targets.interval_list --BAIT_INTERVALS targets.interval_list \
```

```
  --REFERENCE_SEQUENCE $REFERENCE   # 新增：靶区深度（§11）

# 4) 标记重复
gatk MarkDuplicates -I S_sorted.bam -O S_sorted.markdup.bam -M S_markdup_metrics.txt --CREATE_INDEX
true

# 5) BQSR
gatk BaseRecalibrator -R $REFERENCE -I S_sorted.markdup.bam -L $TARGETS \
  --known-sites $DBSNP --known-sites $INDELS --known-sites $HIGHCONF -O S.recal.table
gatk ApplyBQSR -R $REFERENCE -I S_sorted.markdup.bam --bqsr-recal-file S.recal.table -O S.BQSR.bam

# 6) 胚系 GVCF 工作流
gatk HaplotypeCaller -ERC GVCF -R $REFERENCE -I S.BQSR.bam -D $DBSNP -L $TARGETS \
  --native-pair-hmm-threads 8 -O S.g.vcf
gatk CombineGVCFs -R $REFERENCE -V OC.g.vcf -V blood.g.vcf -O combined.g.vcf
gatk GenotypeGVCFs -R $REFERENCE -V combined.g.vcf -D $DBSNP -O joint.vcf

# 7) 过滤 —— 小变异集用硬过滤（VQSR 需要 ~30+ 样本）
gatk VariantFiltration -V joint.vcf -O joint.filtered.vcf \
  --filter-name SNP_QD2 --filter-expression "QD < 2.0 && SNP" \
  --filter-name SNP_MQ40 --filter-expression "MQ < 40.0 && SNP" \
  --filter-name SNP_FS60 --filter-expression "FS > 60.0 && SNP" \
  --filter-name SNP_SOR3 --filter-expression "SOR > 3.0 && SNP" \
  --filter-name INDEL_QD2 --filter-expression "QD < 2.0 && INDEL" \
  --filter-name INDEL_FS200 --filter-expression "FS > 200.0 && INDEL" \
  --filter-name INDEL_SOR10 --filter-expression "SOR > 10.0 && INDEL"
# （队列规模才用 VQSR：VariantRecalibrator SNP → ApplyVQSR → VariantRecalibrator INDEL
#  → 对 SNP 输出再 ApplyVQSR——顺序套用，永远不要合并两个完整变异集）

# 8) ANNOVAR（实际安装的数据库）
perl $ANNOVAR/convert2annovar.pl -format vcf4 joint.filtered.vcf -includeinfo -comment \
  -out ann.avinput
perl $ANNOVAR/table_annovar.pl ann.avinput $HUMANDB -buildver hg38 -out anno \
  -remove -protocol refGeneWithVer,cytoBand,gnomad41_exome,dbnsfp54a,avsnp151 \
  -operation g,r,f,f,f

# 9) 体细胞 TMB（加上 PoN）
gatk Mutect2 -R $REFERENCE -I OC.BQSR.bam -I blood.BQSR.bam --normal-sample blood \
  -L $TARGETS --germline-resource $HIGHCONF --pon $PON \
  -O somatic.raw.vcf.gz --native-pair-hmm-threads 8 --f1r2-tar-gz f1r2.tar.gz
gatk LearnReadOrientationModel -I f1r2.tar.gz -O ob.tar.gz
gatk FilterMutectCalls -R $REFERENCE -V somatic.raw.vcf.gz --stats somatic.raw.vcf.gz.stats \
  --ob-priors ob.tar.gz -O somatic.filtered.vcf.gz
TMB=$(bcftools view -H -f PASS somatic.filtered.vcf.gz | wc -l)
echo "TMB = $(awk -v n=$TMB -v mb=60.5079 'BEGIN{printf "%.2f", n/mb}') mut/Mb"

# 10) CNVkit（hybrid 模式）
cnvkit.py batch OC.BQSR.bam -n blood.BQSR.bam -f $REFERENCE -t $TARGETS \
  -g $ACCESS -m hybrid --output-reference reference.cnn -d cnvkit_out \
  --scatter --diagram -p 8
```

由 §13 所述的两轮实跑生成；文中全部数字均实测。中文版与英文矫正版（*WES_roadmap_corrected.html/pdf*）内容一致、互为对照。